

TASK 1: Business Understanding

1. Clearly define the problem statement.
2. Identify:
 - a. Which is more costly? False Positive or False Negative?
3. Choose the most appropriate evaluation metric and justify your choice.

1. Defining problem statement

The objective of this project is to develop a ML model that predicts whether a credit card customer will miss the next month's payment. This helps financial institutions identify high-risk customers early and take preventive actions to minimize financial losses.

2. Identify:

a. Which is more costly? False Positive or False Negative?

****False Negative (more dangerous-Financial loss)**

Actual situation: Customer misses payment, (High risk=1)

Model prediction: Customer is safe

****False Positive (less dangerous-Customer Dissatisfaction)**

Actual situation: Customer pays normally, (low risk=0)

Model prediction: Customer is risky

False negatives are more costly because predicting a risky customer as low risk may lead to financial loss for the bank. False positives may only cause minor customer dissatisfaction.

3. Choose the most appropriate evaluation metric and justify your choice

Recall is chosen as the primary evaluation metric because it measures the model's ability to correctly identify risky customers and minimize false negatives.

TASK 2: Data Understanding & EDA

1. Dataset shape, data types
2. Check missing values
3. Class distribution (Target Variable) (Is it imbalanced?)
4. Visual analysis:

- a. default payment next month rate by gender
- b. default payment next month by education
- c. Age distribution
- d. Payment delay distribution

5. Correlation analysis

```
In [27]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [28]: bill_payments=pd.read_csv('credi bill payments.xls - Data.csv')
bill_payments.head()
```

Out[28]:

	LIMIT_BAL	GENDER	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	F
0	20000	2	2	1	24	2	2	-1	-1	
1	120000	2	2	2	26	-1	2	0	0	
2	90000	2	2	2	34	0	0	0	0	
3	50000	2	2	1	37	0	0	0	0	
4	50000	1	2	1	57	-1	0	-1	0	

5 rows × 24 columns



```
In [29]: bill_payments.columns
```

```
Out[29]: Index(['LIMIT_BAL', 'GENDER', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0', 'PAY_2',
              'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1', 'BILL_AMT2',
              'BILL_AMT3', 'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1',
              'PAY_AMT2', 'PAY_AMT3', 'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6',
              'default payment next month'],
              dtype='object')
```

```
In [30]: # Dataset shape, data types
bill_payments.shape
```

Out[30]: (30000, 24)

```
In [31]: bill_payments.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   LIMIT_BAL                             30000 non-null  int64
1   GENDER                                30000 non-null  int64
2   EDUCATION                             30000 non-null  int64
3   MARRIAGE                              30000 non-null  int64
4   AGE                                    30000 non-null  int64
5   PAY_0                                 30000 non-null  int64
6   PAY_2                                 30000 non-null  int64
7   PAY_3                                 30000 non-null  int64
8   PAY_4                                 30000 non-null  int64
9   PAY_5                                 30000 non-null  int64
10  PAY_6                                 30000 non-null  int64
11  BILL_AMT1                             30000 non-null  int64
12  BILL_AMT2                             30000 non-null  int64
13  BILL_AMT3                             30000 non-null  int64
14  BILL_AMT4                             30000 non-null  int64
15  BILL_AMT5                             30000 non-null  int64
16  BILL_AMT6                             30000 non-null  int64
17  PAY_AMT1                              30000 non-null  int64
18  PAY_AMT2                              30000 non-null  int64
19  PAY_AMT3                              30000 non-null  int64
20  PAY_AMT4                              30000 non-null  int64
21  PAY_AMT5                              30000 non-null  int64
22  PAY_AMT6                              30000 non-null  int64
23  default payment next month            30000 non-null  int64
dtypes: int64(24)
memory usage: 5.5 MB

```

```

In [32]: #Check missing values
bill_payments.isnull().sum()

```

```

Out[32]: LIMIT_BAL                0
GENDER                0
EDUCATION              0
MARRIAGE               0
AGE                    0
PAY_0                  0
PAY_2                  0
PAY_3                  0
PAY_4                  0
PAY_5                  0
PAY_6                  0
BILL_AMT1              0
BILL_AMT2              0
BILL_AMT3              0
BILL_AMT4              0
BILL_AMT5              0
BILL_AMT6              0
PAY_AMT1               0
PAY_AMT2               0
PAY_AMT3               0
PAY_AMT4               0
PAY_AMT5               0
PAY_AMT6               0
default payment next month  0
dtype: int64

```

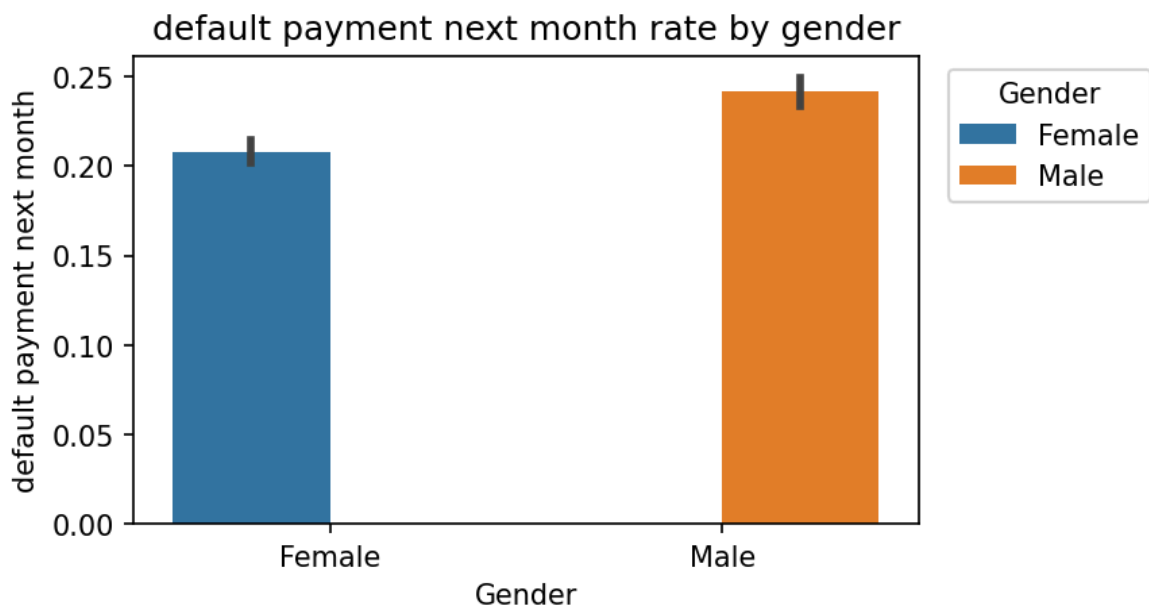
```
In [33]: #Class distribution (Target Variable) (Is it imbalanced?)
bill_payments['default payment next month'].value_counts()
```

```
Out[33]: default payment next month
0      23364
1       6636
Name: count, dtype: int64
```

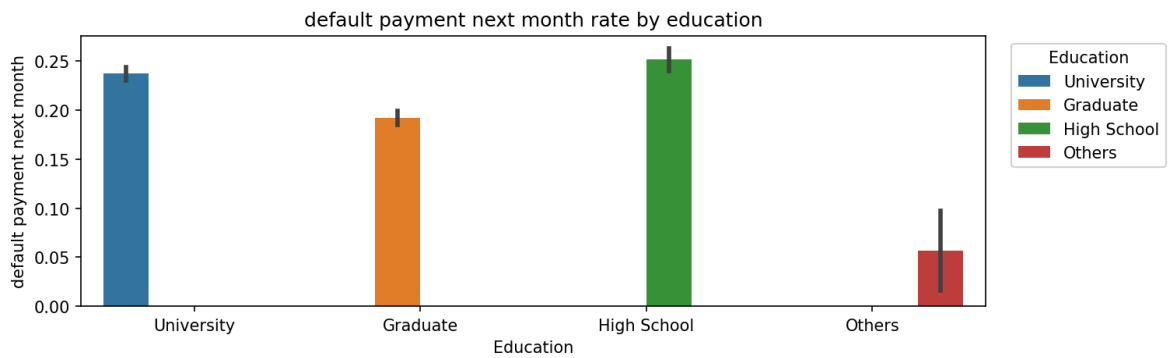
Is it imbalanced?

yes, the given dataset is imbalanced

```
In [35]: # Visual analysis
#a) default payment next month rate by gender
bill_payments["GENDER"] = bill_payments["GENDER"].map({1: "Male", 2: "Female"})
plt.figure(figsize=(5,3),dpi=150)
sns.barplot(x="GENDER", y="default payment next month", hue="GENDER", data=bill_pa
plt.title("default payment next month rate by gender")
plt.xlabel("Gender")
plt.legend(title="Gender", bbox_to_anchor=(1.02,1), loc="upper left");
plt.ylabel("default payment next month");
```

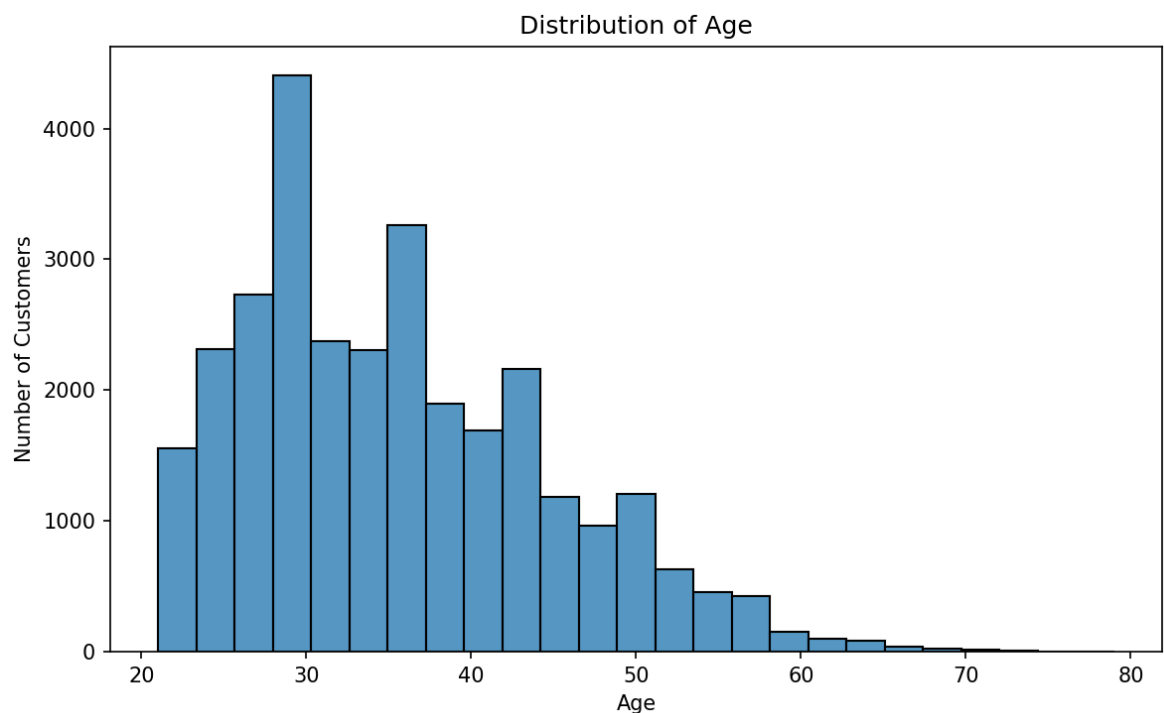


```
In [36]: #default payment next month by education
bill_payments["EDUCATION"] = bill_payments["EDUCATION"].map({1:"Graduate", 2:"Un
plt.figure(figsize=(10,3),dpi=150)
sns.barplot(x="EDUCATION", y="default payment next month", hue="EDUCATION", data=b
plt.title("default payment next month rate by education")
plt.xlabel("Education")
plt.legend(title="Education", bbox_to_anchor=(1.02,1), loc="upper left");
plt.ylabel("default payment next month");
```



```
In [37]: #Age distribution
plt.figure(figsize=(8,5),dpi=150)
sns.histplot(data=bill_payments,bins=25,x="AGE");
plt.title("Distribution of Age")
plt.xlabel("Age")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
```

C:\Users\gunap\anaconda3\Lib\site-packages\seaborn_oldcore.py:1119: FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
with pd.option_context('mode.use_inf_as_na', True):

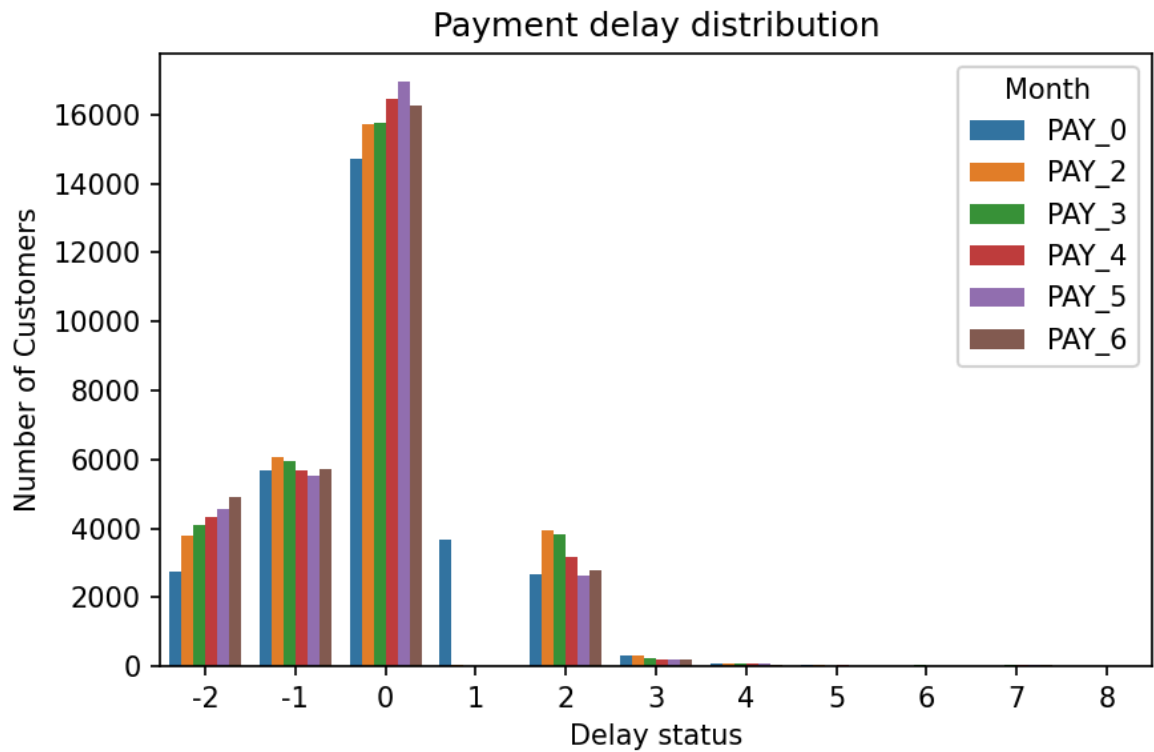


```
In [38]: #Payment delay distribution
pay_cols=['PAY_0','PAY_2','PAY_3','PAY_4','PAY_5','PAY_6']
delay_data=bill_payments[pay_cols].melt(var_name="Month", value_name="Delay_Stat")
plt.figure(figsize=(6,4),dpi=150)
sns.countplot(x="Delay_Status", hue="Month", data=delay_data)
plt.title("Payment delay distribution")
plt.xlabel("Delay status")
plt.ylabel("Number of Customers")
plt.tight_layout()
plt.show()
#Delay Value      Meaning
#-2                No credit card usage / no bill generated
```

```

#-1      Paid on time
#1       1 month payment delay
#2       2 months payment delay
#3       3 months delay
#4+      Longer payment delays

```



```

In [39]: #Correlation analysis
bill_payments["GENDER"] = bill_payments["GENDER"].map({ "Male":1, "Female":2}).
bill_payments["EDUCATION"] = bill_payments["EDUCATION"].map({
    "Graduate":1,
    "University":2,
    "High School":3,
    "Others":4
}).astype("Int64")
correlation=bill_payments.corr()
correlation

```

Out[39]:

	LIMIT_BAL	GENDER	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_1
LIMIT_BAL	1.000000	0.024755	-0.244360	-0.108139	0.144713	-0.271214	-0.296382
GENDER	0.024755	1.000000	0.012584	-0.031389	-0.090874	-0.057643	-0.070771
EDUCATION	-0.244360	0.012584	1.000000	-0.153509	0.187570	0.123181	0.142972
MARRIAGE	-0.108139	-0.031389	-0.153509	1.000000	-0.414170	0.019917	0.024199
AGE	0.144713	-0.090874	0.187570	-0.414170	1.000000	-0.039447	-0.050148
PAY_0	-0.271214	-0.057643	0.123181	0.019917	-0.039447	1.000000	0.672164
PAY_1	-0.296382	-0.070771	0.142972	0.024199	-0.050148	0.672164	1.000000
PAY_2	-0.286123	-0.066096	0.136138	0.032688	-0.053048	0.574245	0.762164
PAY_3	-0.267460	-0.060173	0.128501	0.033122	-0.049722	0.538841	0.662164
PAY_4	-0.249411	-0.055064	0.115204	0.035629	-0.053826	0.509426	0.622164
PAY_5	-0.235195	-0.044008	0.102788	0.034345	-0.048773	0.474553	0.572164
BILL_AMT1	0.285430	-0.033642	0.004191	-0.023472	0.056239	0.187068	0.232164
BILL_AMT2	0.278314	-0.031183	0.000512	-0.021602	0.054283	0.189859	0.232164
BILL_AMT3	0.283236	-0.024563	-0.004419	-0.024909	0.053710	0.179785	0.222164
BILL_AMT4	0.293988	-0.021880	-0.015327	-0.023344	0.051353	0.179125	0.222164
BILL_AMT5	0.295562	-0.017005	-0.019113	-0.025393	0.049345	0.180635	0.222164
BILL_AMT6	0.290389	-0.016733	-0.016165	-0.021207	0.047613	0.176980	0.212164
PAY_AMT1	0.195236	-0.000242	-0.045338	-0.005979	0.026147	-0.079269	-0.082164
PAY_AMT2	0.178408	-0.001391	-0.038943	-0.008093	0.021785	-0.070101	-0.052164
PAY_AMT3	0.210167	-0.008597	-0.052136	-0.003541	0.029247	-0.070561	-0.052164
PAY_AMT4	0.203242	-0.002229	-0.043463	-0.012659	0.021379	-0.064005	-0.042164
PAY_AMT5	0.217202	-0.001667	-0.049799	-0.001205	0.022850	-0.058190	-0.032164
PAY_AMT6	0.219595	-0.002766	-0.053887	-0.006641	0.019478	-0.058673	-0.032164
default payment next month	-0.153520	-0.039961	0.048248	-0.024339	0.013890	0.324794	0.262164

24 rows × 24 columns



In [40]: `bill_payments.corr()["default payment next month"].sort_values(ascending=False)`

```

Out[40]: default payment next month    1.000000
PAY_0                                0.324794
PAY_2                                0.263551
PAY_3                                0.235253
PAY_4                                0.216614
PAY_5                                0.204149
PAY_6                                0.186866
EDUCATION                            0.048248
AGE                                   0.013890
BILL_AMT6                            -0.005372
BILL_AMT5                            -0.006760
BILL_AMT4                            -0.010156
BILL_AMT3                            -0.014076
BILL_AMT2                            -0.014193
BILL_AMT1                            -0.019644
MARRIAGE                             -0.024339
GENDER                               -0.039961
PAY_AMT6                             -0.053183
PAY_AMT5                             -0.055124
PAY_AMT3                             -0.056250
PAY_AMT4                             -0.056827
PAY_AMT2                             -0.058579
PAY_AMT1                             -0.072929
LIMIT_BAL                           -0.153520
Name: default payment next month, dtype: float64

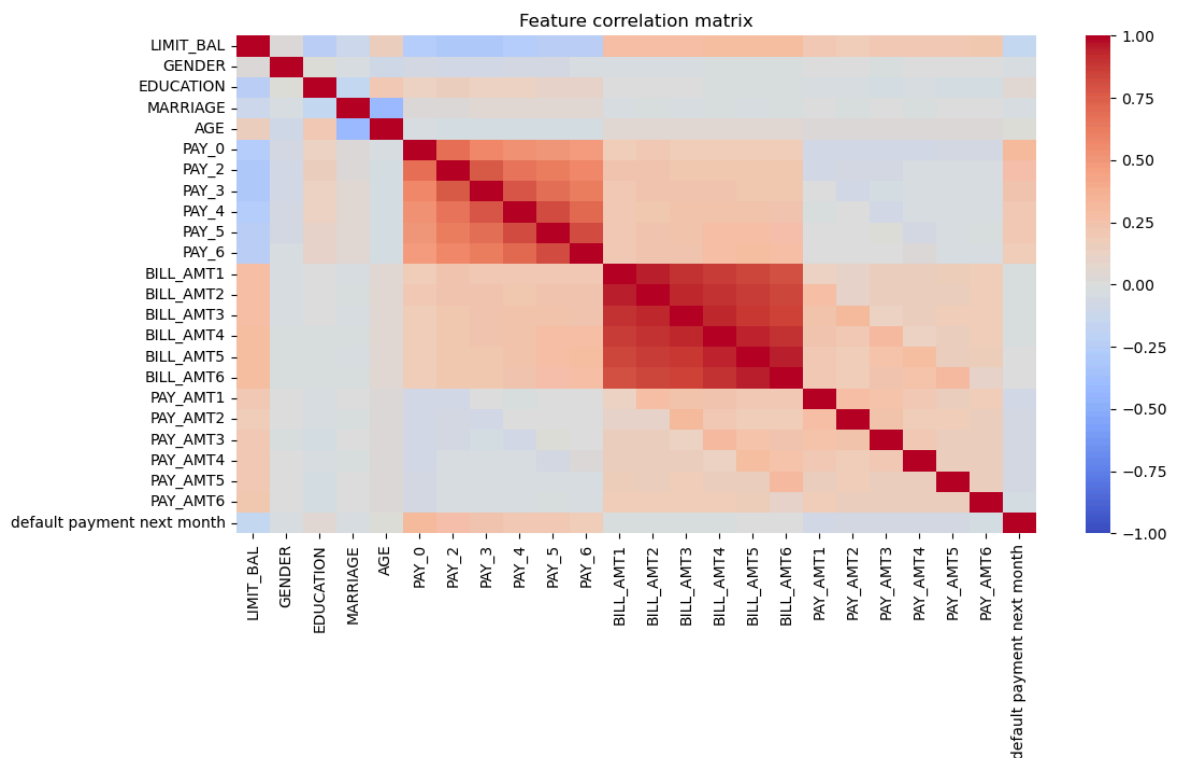
```

Default payment next month is highly correlated with PAY_0

```

In [42]: plt.figure(figsize=(12,6))
sns.heatmap(correlation,cmap='coolwarm',fmt='.2f',vmin=-1,vmax=1)
plt.title("Feature correlation matrix" )
plt.show()

```



TASK 3: Data Preprocessing

1. Handle invalid or unusual values (if any)
2. Encoding (if needed)
3. Apply scaling where necessary
4. Train-test split

In [44]: `bill_payments.head()`

Out[44]:

	LIMIT_BAL	GENDER	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	F
0	20000	2	2	1	24	2	2	-1	-1	
1	120000	2	2	2	26	-1	2	0	0	
2	90000	2	2	2	34	0	0	0	0	
3	50000	2	2	1	37	0	0	0	0	
4	50000	1	2	1	57	-1	0	-1	0	

5 rows × 24 columns



In [45]: `#Handle invalid or unusual values (if any)`
`for col in bill_payments.select_dtypes(include=["int64","float64"]):`
 `print(col, bill_payments[col].min(), bill_payments[col].max())`

```
LIMIT_BAL 10000 1000000
GENDER 1 2
EDUCATION 1 4
MARRIAGE 0 3
AGE 21 79
PAY_0 -2 8
PAY_2 -2 8
PAY_3 -2 8
PAY_4 -2 8
PAY_5 -2 8
PAY_6 -2 8
BILL_AMT1 -165580 964511
BILL_AMT2 -69777 983931
BILL_AMT3 -157264 1664089
BILL_AMT4 -170000 891586
BILL_AMT5 -81334 927171
BILL_AMT6 -339603 961664
PAY_AMT1 0 873552
PAY_AMT2 0 1684259
PAY_AMT3 0 896040
PAY_AMT4 0 621000
PAY_AMT5 0 426529
PAY_AMT6 0 528666
default payment next month 0 1
```

In [46]: `bill_payments["EDUCATION"].fillna(bill_payments["EDUCATION"].mode()[0], inplace=`

In [47]: `#handling marriage data according to dataset description`
`bill_payments['MARRIAGE'].unique()`

```
Out[47]: array([1, 2, 3, 0], dtype=int64)
```

```
In [48]: bill_payments['MARRIAGE'].replace(0,3,inplace=True)
bill_payments['MARRIAGE'].unique()
```

```
Out[48]: array([1, 2, 3], dtype=int64)
```

```
In [49]: pay_cols = ['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']
for col in pay_cols:
    print(col, bill_payments[col].unique())
```

```
PAY_0 [ 2 -1  0 -2  1  3  4  8  7  5  6]
PAY_2 [ 2  0 -1 -2  3  5  7  4  1  6  8]
PAY_3 [-1  0  2 -2  3  4  6  7  1  5  8]
PAY_4 [-1  0 -2  2  3  4  5  7  6  1  8]
PAY_5 [-2  0 -1  2  3  5  4  7  8  6]
PAY_6 [-2  2  0 -1  3  6  4  7  8  5]
```

```
In [50]: for col in pay_cols:
    bill_payments[col].replace(-2,-1,inplace=True)
```

```
In [51]: for col in pay_cols:
    print(col, bill_payments[col].unique())
```

```
PAY_0 [ 2 -1  0  1  3  4  8  7  5  6]
PAY_2 [ 2  0 -1  3  5  7  4  1  6  8]
PAY_3 [-1  0  2  3  4  6  7  1  5  8]
PAY_4 [-1  0  2  3  4  5  7  6  1  8]
PAY_5 [-1  0  2  3  5  4  7  8  6]
PAY_6 [-1  2  0  3  6  4  7  8  5]
```

Encoding (if needed)

No encoding is required

Apply scaling where necessary

Scaling is not necessary for all models of the classification.

Scaling is required for :

a) Logistic Regression

b) KNN

c) SVM

These algorithms depend on distance or gradient calculations, so scaling improves their performance.

Models that do not require scaling:

a) Decision Tree

b) Random Forest

- c) AdaBoost
- d) XGBoost
- e) LightGBM
- f) CatBoost

Tree-based models split data by thresholds and are not sensitive to feature scale.

```
In [54]: # Train-test split
X=bill_payments.drop('default payment next month',axis=1)
y=bill_payments['default payment next month']
```

```
In [55]: y.head()
```

```
Out[55]: 0    1
         1    1
         2    0
         3    0
         4    0
         Name: default payment next month, dtype: int64
```

```
In [56]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=10)
```

TASK 4: Models Building

1. Build the following models using default parameters:

- a. Logistic Regression
- b. KNN
- c. SVM
- d. Decision Tree
- e. Random Forest
- f. AdaBoost
- g. XGBoost
- h. LightGBM
- i. CatBoost

2. Evaluate each using:

- a. Accuracy
- b. Precision
- c. Recall

d. F1 Score

```
In [58]: #Build the following models using default parameters  
# a)Logistic Regression  
# Feature Scaling  
from sklearn.preprocessing import StandardScaler  
scaler=StandardScaler()
```

```
In [59]: # scale features  
X_train_scaled=scaler.fit_transform(X_train)  
X_test_scaled=scaler.transform(X_test)
```

```
In [60]: X_train_scaled[:10]
```

```
Out[60]: array([[ -1.06533126,  0.80833973,  0.25385171, -1.06672953, -0.16393338 ,
-1.0796122 , -0.95264732,  1.95410948,  0.07319354,  0.11989307,
 0.13462474, -0.68964877, -0.67782455, -0.6669585 , -0.65143036,
-0.64084394, -0.62874495, -0.27769218, -0.23989221, -0.2118446 ,
-0.28706816, -0.28350864, -0.29194625],
 [ 0.3216164 ,  0.80833973,  1.6645122 ,  0.84987897,  1.13831752,
 0.92665798,  1.92765124,  0.02516809,  0.07319354, -0.93312007,
-0.91168152, -0.61648108, -0.61365656, -0.38832922, -0.40588145,
-0.62620654, -0.64799932, -0.33468939,  0.44962638, -0.28899898,
-0.16379685, -0.29364408, -0.07136759],
 [-0.68006802,  0.80833973, -1.15680878,  0.84987897, -0.81505947,
-0.07647711,  0.0074522 ,  0.02516809,  0.07319354,  0.11989307,
 0.13462474,  0.26216745,  0.31823515,  0.38053043,  0.49157281,
 0.56193831,  0.59962404, -0.15064671, -0.10046676, -0.10969655,
-0.13217228, -0.12414637, -0.13202672],
 [ 0.55277434,  0.80833973, -1.15680878,  0.84987897,  0.05310808,
 0.92665798, -0.95264732,  0.02516809,  0.07319354,  0.11989307,
 0.13462474, -0.69521249, -0.43978426, -0.41110538, -0.40438147,
-0.34791632, -0.30030564,  0.70402279, -0.16022052, -0.18033084,
-0.18380424, -0.11777188, -0.18165692],
 [-0.60301537,  0.80833973,  0.25385171, -1.06672953, -0.27245475,
-0.07647711,  0.0074522 ,  0.02516809,  0.07319354,  0.11989307,
-0.91168152, -0.64354161, -0.57213774, -0.54719078, -0.49298876,
-0.64451962, -0.65200995, -0.04670042, -0.18870315, -0.12599677,
-0.23614614, -0.3090066 ,  0.19509142],
 [-0.14069949,  0.80833973, -1.15680878,  0.84987897, -0.92358041,
 1.92979307,  1.92765124,  1.95410948,  2.07849593,  0.11989307,
 0.13462474, -0.48572683, -0.43681663, -0.40921927, -0.39488673,
-0.35802854, -0.32892926, -0.1621963 , -0.20005636, -0.28899898,
-0.24834419, -0.24526169, -0.23680158],
 [ 0.3216164 ,  0.80833973,  0.25385171,  0.84987897, -0.92358041,
-0.07647711,  0.0074522 ,  0.02516809,  0.07319354,  0.11989307,
 0.13462474, -0.0716153 , -0.14649286, -0.25035771, -0.43569556,
-0.46014725, -0.47669033, -0.22277389, -0.11903026, -0.12599677,
-0.11926429, -0.11777188, -0.28687294],
 [-0.68006802,  0.80833973,  1.6645122 ,  0.84987897, -0.48949664,
-0.07647711,  0.0074522 ,  0.02516809,  2.07849593,  2.22591935,
 2.22723727, -0.22656023, -0.19118928, -0.10339863, -0.04314824,
-0.01078247,  0.09055643, -0.21994424, -0.03274583, -0.19663106,
-0.31288414,  0.04159039, -0.29194625],
 [-0.75712067,  0.80833973, -1.15680878, -1.06672953,  1.02979657,
 0.92665798,  1.92765124,  1.95410948,  0.07319354,  0.11989307,
 0.13462474,  0.24959154,  0.29870758,  0.25850509,  0.22726637,
-0.0767651 , -0.18720909, -0.1621963 , -0.23989221, -0.12599677,
 0.07067679,  0.32844246, -0.1679259 ],
 [-0.44891008,  0.80833973,  0.25385171,  0.84987897, -1.24914324,
-0.07647711,  0.0074522 ,  0.02516809,  0.07319354,  0.11989307,
 0.13462474,  0.64282896,  0.60756504,  0.44260046,  0.40004286,
 0.49572697,  0.61000842, -0.15064671, -0.10046676, -0.12599677,
-0.08699431,  0.13720774, -0.12651226]])
```

LogisticRegression (Train the model)

```
In [62]: from sklearn.linear_model import LogisticRegression
logistic_M1=LogisticRegression()
logistic_M1.fit(X_train_scaled,y_train)
```

```
Out[62]: ▾ LogisticRegression
LogisticRegression()
```

```
In [63]: y_pred=logistic_M1.predict(X_test_scaled)
y_pred[:10]
```

```
Out[63]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
In [64]: y_test[:10]
```

```
Out[64]: 6015      1
28660     0
12498     0
22957     0
14848     0
21298     0
3798      0
21714     0
8362      1
14399     0
Name: default payment next month, dtype: int64
```

```
In [65]: ## model evaluation
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
In [66]: confusion_matrix(y_test, y_pred)
```

```
Out[66]: array([[6853, 205],
               [1429, 513]], dtype=int64)
```

```
In [67]: accuracy_score(y_test, y_pred)
```

```
Out[67]: 0.8184444444444444
```

```
In [68]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.97	0.89	7058
1	0.71	0.26	0.39	1942
accuracy			0.82	9000
macro avg	0.77	0.62	0.64	9000
weighted avg	0.80	0.82	0.78	9000

KNN

```
In [70]: from sklearn.neighbors import KNeighborsClassifier
```

```
In [71]: KNN_M2=KNeighborsClassifier()
```

```
In [72]: KNN_M2.fit(X_train_scaled, y_train)
```

```
Out[72]: ▾ KNeighborsClassifier
KNeighborsClassifier()
```

```
In [73]: y_pred_2=KNN_M2.predict(X_test_scaled)
y_pred_2[:10]
```

```
Out[73]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
In [74]: y_test[:10]
```

```
Out[74]: 6015      1
28660      0
12498      0
22957      0
14848      0
21298      0
3798       0
21714      0
8362       1
14399      0
Name: default payment next month, dtype: int64
```

```
In [75]: ## model evaluation
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
In [76]: confusion_matrix(y_test, y_pred_2)
```

```
Out[76]: array([[6467,  591],
               [1278,  664]], dtype=int64)
```

```
In [77]: accuracy_score(y_test, y_pred_2)
```

```
Out[77]: 0.7923333333333333
```

```
In [78]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.97	0.89	7058
1	0.71	0.26	0.39	1942
accuracy			0.82	9000
macro avg	0.77	0.62	0.64	9000
weighted avg	0.80	0.82	0.78	9000

SVM

```
In [80]: from sklearn.svm import SVC
```

```
In [81]: svm_M3=SVC()
```

```
In [82]: svm_M3.fit(X_train_scaled, y_train)
```

Out[82]:  SVC
SVC()

```
In [83]: y_pred_3=svm_M3.predict(X_test_scaled)
y_pred_3[:3]
```

Out[83]: array([0, 0, 0], dtype=int64)

```
In [84]: #model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_repor
```

```
In [85]: confusion_matrix(y_test,y_pred_3)
```

Out[85]: array([[6774, 284],
[1295, 647]], dtype=int64)

```
In [86]: accuracy_score(y_test,y_pred_3)
```

Out[86]: 0.8245555555555556

```
In [87]: print(classification_report(y_test,y_pred_3))
```

	precision	recall	f1-score	support
0	0.84	0.96	0.90	7058
1	0.69	0.33	0.45	1942
accuracy			0.82	9000
macro avg	0.77	0.65	0.67	9000
weighted avg	0.81	0.82	0.80	9000

Decision Tree

```
In [89]: from sklearn.tree import DecisionTreeClassifier
```

```
In [90]: Decision_tree_M4=DecisionTreeClassifier()
Decision_tree_M4.fit(X_train,y_train)
```

Out[90]:  DecisionTreeClassifier
DecisionTreeClassifier()

```
In [91]: y_pred_4=Decision_tree_M4.predict(X_test)
y_pred_4[:10]
```

Out[91]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)

```
In [92]: # model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_repor
```

```
In [93]: confusion_matrix(y_test,y_pred_4)
```



```
Out[93]: array([[5749, 1309],
               [1125,  817]], dtype=int64)
```

```
In [94]: accuracy_score(y_test,y_pred_4)
```

```
Out[94]: 0.7295555555555555
```

```
In [95]: print(classification_report(y_test,y_pred_4))
```

	precision	recall	f1-score	support
0	0.84	0.81	0.83	7058
1	0.38	0.42	0.40	1942
accuracy			0.73	9000
macro avg	0.61	0.62	0.61	9000
weighted avg	0.74	0.73	0.73	9000

Random Forest

```
In [97]: from sklearn.ensemble import RandomForestClassifier
```

```
In [98]: Random_forest_M5=RandomForestClassifier()
```

```
In [99]: Random_forest_M5.fit(X_train,y_train)
```

```
Out[99]: ▼ RandomForestClassifier
RandomForestClassifier()
```

```
In [100... y_pred_5=Random_forest_M5.predict(X_test)
y_pred_5[:5]
```

```
Out[100... array([0, 0, 0, 0, 0], dtype=int64)
```

```
In [101... # model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_repor
```

```
In [102... confusion_matrix(y_test,y_pred_5)
```

```
Out[102... array([[6648,  410],
               [1238,  704]], dtype=int64)
```

```
In [103... accuracy_score(y_test,y_pred_5)
```

```
Out[103... 0.8168888888888889
```

```
In [104... print(classification_report(y_test,y_pred_5))
```

	precision	recall	f1-score	support
0	0.84	0.94	0.89	7058
1	0.63	0.36	0.46	1942
accuracy			0.82	9000
macro avg	0.74	0.65	0.68	9000
weighted avg	0.80	0.82	0.80	9000

AdaBoost

```
In [106... from sklearn.ensemble import AdaBoostClassifier
```

```
In [107... AdaBoost_M6=AdaBoostClassifier()
```

```
In [108... AdaBoost_M6.fit(X_train,y_train)
```

```
Out[108... ▼ AdaBoostClassifier
AdaBoostClassifier()
```

```
In [109... y_pred_6=AdaBoost_M6.predict(X_test)
y_pred_6[:10]
```

```
Out[109... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
In [110... # Model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_repor
```

```
In [111... confusion_matrix(y_test,y_pred_6)
```

```
Out[111... array([[6778, 280],
       [1332, 610]], dtype=int64)
```

```
In [112... accuracy_score(y_test,y_pred_6)
```

```
Out[112... 0.8208888888888889
```

```
In [113... print(classification_report(y_test,y_pred_6))
```

	precision	recall	f1-score	support
0	0.84	0.96	0.89	7058
1	0.69	0.31	0.43	1942
accuracy			0.82	9000
macro avg	0.76	0.64	0.66	9000
weighted avg	0.80	0.82	0.79	9000

XGBoost

```
In [115... from xgboost import XGBClassifier
```

```
In [116... XGB_M7=XGBClassifier()
```

```
In [117... XGB_M7.fit(X_train,y_train)
```

```
Out[117... XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=None,
               enable_categorical=False, eval_metric=None, feature_types=None,
               gamma=None, grow_policy=None, importance_type=None,
               interaction_constraints=None, learning_rate=None, max_bin=None,
```

```
In [118... y_pred_7=XGB_M7.predict(X_test)
y_pred_7
```

```
Out[118... array([0, 0, 0, ..., 0, 0, 0])
```

```
In [119... # Model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_report
```

```
In [120... confusion_matrix(y_test,y_pred_7)
```

```
Out[120... array([[6612,  446],
       [1244,  698]], dtype=int64)
```

```
In [121... accuracy_score(y_test,y_pred_7)
```

```
Out[121... 0.8122222222222222
```

```
In [122... print(classification_report(y_test,y_pred_7))
```

	precision	recall	f1-score	support
0	0.84	0.94	0.89	7058
1	0.61	0.36	0.45	1942
accuracy			0.81	9000
macro avg	0.73	0.65	0.67	9000
weighted avg	0.79	0.81	0.79	9000

LightGBM

```
In [124... from lightgbm import LGBMClassifier
```

```
In [125... LGBM_M8=LGBMClassifier()
```

```
In [126... LGBM_M8.fit(X_train,y_train)
```

```
[LightGBM] [Info] Number of positive: 4694, number of negative: 16306
[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing
was 0.001454 seconds.
You can set `force_col_wise=true` to remove the overhead.
[LightGBM] [Info] Total Bins 3256
[LightGBM] [Info] Number of data points in the train set: 21000, number of used f
eatures: 23
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.223524 -> initscore=-1.245248
[LightGBM] [Info] Start training from score -1.245248
```

```
Out[126... ▾ LGBMClassifier
LGBMClassifier()
```

```
In [127... y_pred_8=LGBM_M8.predict(X_test)
y_pred_8[:10]
```

```
Out[127... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
In [128... # Model Evaluation
from sklearn.metrics import confusion_matrix, accuracy_score, classification_repor
```

```
In [129... confusion_matrix(y_test, y_pred_8)
```

```
Out[129... array([[6705, 353],
       [1230, 712]], dtype=int64)
```

```
In [130... accuracy_score(y_test, y_pred_8)
```

```
Out[130... 0.8241111111111111
```

```
In [131... print(classification_report(y_test, y_pred_8))
```

	precision	recall	f1-score	support
0	0.84	0.95	0.89	7058
1	0.67	0.37	0.47	1942
accuracy			0.82	9000
macro avg	0.76	0.66	0.68	9000
weighted avg	0.81	0.82	0.80	9000

CatBoost

```
In [133... from catboost import CatBoostClassifier
```

```
In [134... cat_boost_M9=CatBoostClassifier()
```

```
In [135... cat_boost_M9.fit(X_train, y_train)
```

Learning rate set to 0.037803

0:	learn: 0.6718674	total: 161ms	remaining: 2m 40s
1:	learn: 0.6523460	total: 169ms	remaining: 1m 24s
2:	learn: 0.6345658	total: 176ms	remaining: 58.5s
3:	learn: 0.6184878	total: 184ms	remaining: 45.7s
4:	learn: 0.6039734	total: 191ms	remaining: 38s
5:	learn: 0.5900613	total: 199ms	remaining: 32.9s
6:	learn: 0.5776597	total: 207ms	remaining: 29.3s
7:	learn: 0.5664447	total: 214ms	remaining: 26.6s
8:	learn: 0.5559342	total: 222ms	remaining: 24.4s
9:	learn: 0.5463973	total: 232ms	remaining: 22.9s
10:	learn: 0.5373615	total: 240ms	remaining: 21.6s
11:	learn: 0.5293902	total: 248ms	remaining: 20.5s
12:	learn: 0.5219685	total: 256ms	remaining: 19.4s
13:	learn: 0.5145924	total: 264ms	remaining: 18.6s
14:	learn: 0.5085364	total: 272ms	remaining: 17.8s
15:	learn: 0.5028626	total: 280ms	remaining: 17.2s
16:	learn: 0.4973818	total: 287ms	remaining: 16.6s
17:	learn: 0.4923150	total: 295ms	remaining: 16.1s
18:	learn: 0.4880618	total: 303ms	remaining: 15.6s
19:	learn: 0.4838809	total: 311ms	remaining: 15.2s
20:	learn: 0.4798107	total: 319ms	remaining: 14.8s
21:	learn: 0.4761926	total: 326ms	remaining: 14.5s
22:	learn: 0.4731984	total: 334ms	remaining: 14.2s
23:	learn: 0.4703543	total: 341ms	remaining: 13.9s
24:	learn: 0.4677456	total: 348ms	remaining: 13.6s
25:	learn: 0.4650992	total: 356ms	remaining: 13.3s
26:	learn: 0.4628089	total: 364ms	remaining: 13.1s
27:	learn: 0.4604007	total: 372ms	remaining: 12.9s
28:	learn: 0.4585603	total: 380ms	remaining: 12.7s
29:	learn: 0.4565987	total: 387ms	remaining: 12.5s
30:	learn: 0.4548516	total: 395ms	remaining: 12.3s
31:	learn: 0.4531713	total: 402ms	remaining: 12.2s
32:	learn: 0.4516405	total: 410ms	remaining: 12s
33:	learn: 0.4502846	total: 418ms	remaining: 11.9s
34:	learn: 0.4489198	total: 426ms	remaining: 11.7s
35:	learn: 0.4475699	total: 434ms	remaining: 11.6s
36:	learn: 0.4464391	total: 441ms	remaining: 11.5s
37:	learn: 0.4452579	total: 449ms	remaining: 11.4s
38:	learn: 0.4440684	total: 457ms	remaining: 11.3s
39:	learn: 0.4430140	total: 465ms	remaining: 11.2s
40:	learn: 0.4419705	total: 472ms	remaining: 11s
41:	learn: 0.4410181	total: 480ms	remaining: 10.9s
42:	learn: 0.4402087	total: 488ms	remaining: 10.9s
43:	learn: 0.4393831	total: 495ms	remaining: 10.8s
44:	learn: 0.4385741	total: 503ms	remaining: 10.7s
45:	learn: 0.4378610	total: 510ms	remaining: 10.6s
46:	learn: 0.4373155	total: 517ms	remaining: 10.5s
47:	learn: 0.4368292	total: 525ms	remaining: 10.4s
48:	learn: 0.4361997	total: 533ms	remaining: 10.3s
49:	learn: 0.4357399	total: 540ms	remaining: 10.3s
50:	learn: 0.4351083	total: 548ms	remaining: 10.2s
51:	learn: 0.4346358	total: 556ms	remaining: 10.1s
52:	learn: 0.4342486	total: 564ms	remaining: 10.1s
53:	learn: 0.4336992	total: 571ms	remaining: 10s
54:	learn: 0.4331229	total: 579ms	remaining: 9.95s
55:	learn: 0.4327451	total: 587ms	remaining: 9.89s
56:	learn: 0.4322450	total: 595ms	remaining: 9.85s
57:	learn: 0.4318541	total: 603ms	remaining: 9.79s
58:	learn: 0.4314637	total: 610ms	remaining: 9.73s

59:	learn: 0.4311374	total: 618ms	remaining: 9.68s
60:	learn: 0.4308579	total: 625ms	remaining: 9.63s
61:	learn: 0.4304387	total: 633ms	remaining: 9.58s
62:	learn: 0.4300730	total: 641ms	remaining: 9.53s
63:	learn: 0.4297952	total: 648ms	remaining: 9.47s
64:	learn: 0.4294448	total: 655ms	remaining: 9.42s
65:	learn: 0.4290726	total: 662ms	remaining: 9.37s
66:	learn: 0.4288387	total: 670ms	remaining: 9.33s
67:	learn: 0.4285086	total: 678ms	remaining: 9.29s
68:	learn: 0.4281843	total: 686ms	remaining: 9.25s
69:	learn: 0.4278645	total: 693ms	remaining: 9.21s
70:	learn: 0.4276922	total: 701ms	remaining: 9.18s
71:	learn: 0.4273961	total: 709ms	remaining: 9.14s
72:	learn: 0.4271498	total: 717ms	remaining: 9.11s
73:	learn: 0.4268213	total: 725ms	remaining: 9.07s
74:	learn: 0.4265379	total: 732ms	remaining: 9.03s
75:	learn: 0.4263005	total: 740ms	remaining: 9s
76:	learn: 0.4261259	total: 748ms	remaining: 8.96s
77:	learn: 0.4259017	total: 756ms	remaining: 8.93s
78:	learn: 0.4256632	total: 763ms	remaining: 8.9s
79:	learn: 0.4253920	total: 771ms	remaining: 8.87s
80:	learn: 0.4251277	total: 779ms	remaining: 8.84s
81:	learn: 0.4250043	total: 786ms	remaining: 8.8s
82:	learn: 0.4247353	total: 794ms	remaining: 8.77s
83:	learn: 0.4245531	total: 803ms	remaining: 8.75s
84:	learn: 0.4243785	total: 812ms	remaining: 8.74s
85:	learn: 0.4242533	total: 820ms	remaining: 8.71s
86:	learn: 0.4241022	total: 828ms	remaining: 8.69s
87:	learn: 0.4239238	total: 836ms	remaining: 8.66s
88:	learn: 0.4237792	total: 844ms	remaining: 8.64s
89:	learn: 0.4235510	total: 853ms	remaining: 8.63s
90:	learn: 0.4233130	total: 862ms	remaining: 8.61s
91:	learn: 0.4232101	total: 870ms	remaining: 8.59s
92:	learn: 0.4230259	total: 879ms	remaining: 8.57s
93:	learn: 0.4228417	total: 889ms	remaining: 8.57s
94:	learn: 0.4226922	total: 900ms	remaining: 8.57s
95:	learn: 0.4225529	total: 909ms	remaining: 8.56s
96:	learn: 0.4223599	total: 919ms	remaining: 8.56s
97:	learn: 0.4221761	total: 929ms	remaining: 8.55s
98:	learn: 0.4219716	total: 939ms	remaining: 8.54s
99:	learn: 0.4217921	total: 949ms	remaining: 8.54s
100:	learn: 0.4216707	total: 959ms	remaining: 8.53s
101:	learn: 0.4215068	total: 969ms	remaining: 8.53s
102:	learn: 0.4213078	total: 979ms	remaining: 8.53s
103:	learn: 0.4211539	total: 989ms	remaining: 8.52s
104:	learn: 0.4209960	total: 1s	remaining: 8.53s
105:	learn: 0.4208707	total: 1.01s	remaining: 8.52s
106:	learn: 0.4207161	total: 1.02s	remaining: 8.51s
107:	learn: 0.4205717	total: 1.03s	remaining: 8.51s
108:	learn: 0.4205174	total: 1.04s	remaining: 8.5s
109:	learn: 0.4203854	total: 1.05s	remaining: 8.49s
110:	learn: 0.4202637	total: 1.06s	remaining: 8.49s
111:	learn: 0.4201304	total: 1.07s	remaining: 8.49s
112:	learn: 0.4199055	total: 1.08s	remaining: 8.49s
113:	learn: 0.4198274	total: 1.09s	remaining: 8.5s
114:	learn: 0.4196332	total: 1.1s	remaining: 8.5s
115:	learn: 0.4195059	total: 1.12s	remaining: 8.51s
116:	learn: 0.4193843	total: 1.13s	remaining: 8.52s
117:	learn: 0.4192480	total: 1.14s	remaining: 8.52s
118:	learn: 0.4191479	total: 1.15s	remaining: 8.52s

119:	learn: 0.4190661	total: 1.16s	remaining: 8.53s
120:	learn: 0.4189214	total: 1.18s	remaining: 8.54s
121:	learn: 0.4187633	total: 1.19s	remaining: 8.55s
122:	learn: 0.4186325	total: 1.2s	remaining: 8.55s
123:	learn: 0.4185555	total: 1.21s	remaining: 8.56s
124:	learn: 0.4184630	total: 1.22s	remaining: 8.57s
125:	learn: 0.4184089	total: 1.24s	remaining: 8.58s
126:	learn: 0.4183487	total: 1.25s	remaining: 8.59s
127:	learn: 0.4181722	total: 1.26s	remaining: 8.6s
128:	learn: 0.4180521	total: 1.27s	remaining: 8.6s
129:	learn: 0.4179084	total: 1.28s	remaining: 8.6s
130:	learn: 0.4177239	total: 1.3s	remaining: 8.61s
131:	learn: 0.4175719	total: 1.31s	remaining: 8.61s
132:	learn: 0.4175148	total: 1.32s	remaining: 8.61s
133:	learn: 0.4173523	total: 1.33s	remaining: 8.62s
134:	learn: 0.4172412	total: 1.34s	remaining: 8.62s
135:	learn: 0.4171776	total: 1.35s	remaining: 8.61s
136:	learn: 0.4170649	total: 1.36s	remaining: 8.6s
137:	learn: 0.4169931	total: 1.38s	remaining: 8.6s
138:	learn: 0.4168700	total: 1.39s	remaining: 8.6s
139:	learn: 0.4167432	total: 1.4s	remaining: 8.59s
140:	learn: 0.4166487	total: 1.41s	remaining: 8.59s
141:	learn: 0.4165497	total: 1.42s	remaining: 8.58s
142:	learn: 0.4164794	total: 1.43s	remaining: 8.57s
143:	learn: 0.4163822	total: 1.44s	remaining: 8.56s
144:	learn: 0.4162588	total: 1.45s	remaining: 8.55s
145:	learn: 0.4162116	total: 1.46s	remaining: 8.54s
146:	learn: 0.4161283	total: 1.47s	remaining: 8.54s
147:	learn: 0.4159800	total: 1.48s	remaining: 8.53s
148:	learn: 0.4158696	total: 1.49s	remaining: 8.52s
149:	learn: 0.4157790	total: 1.5s	remaining: 8.51s
150:	learn: 0.4157082	total: 1.51s	remaining: 8.5s
151:	learn: 0.4155327	total: 1.52s	remaining: 8.49s
152:	learn: 0.4154658	total: 1.53s	remaining: 8.48s
153:	learn: 0.4153947	total: 1.54s	remaining: 8.47s
154:	learn: 0.4153431	total: 1.55s	remaining: 8.46s
155:	learn: 0.4152773	total: 1.56s	remaining: 8.45s
156:	learn: 0.4151933	total: 1.57s	remaining: 8.44s
157:	learn: 0.4150556	total: 1.58s	remaining: 8.43s
158:	learn: 0.4149636	total: 1.59s	remaining: 8.42s
159:	learn: 0.4148668	total: 1.6s	remaining: 8.41s
160:	learn: 0.4147503	total: 1.61s	remaining: 8.4s
161:	learn: 0.4146986	total: 1.62s	remaining: 8.39s
162:	learn: 0.4146098	total: 1.63s	remaining: 8.37s
163:	learn: 0.4145380	total: 1.64s	remaining: 8.36s
164:	learn: 0.4143765	total: 1.65s	remaining: 8.35s
165:	learn: 0.4142918	total: 1.66s	remaining: 8.34s
166:	learn: 0.4141696	total: 1.67s	remaining: 8.33s
167:	learn: 0.4140653	total: 1.68s	remaining: 8.32s
168:	learn: 0.4139359	total: 1.69s	remaining: 8.31s
169:	learn: 0.4138427	total: 1.7s	remaining: 8.3s
170:	learn: 0.4137495	total: 1.71s	remaining: 8.29s
171:	learn: 0.4136460	total: 1.72s	remaining: 8.28s
172:	learn: 0.4135747	total: 1.73s	remaining: 8.27s
173:	learn: 0.4134654	total: 1.74s	remaining: 8.25s
174:	learn: 0.4133045	total: 1.75s	remaining: 8.24s
175:	learn: 0.4132238	total: 1.76s	remaining: 8.23s
176:	learn: 0.4131372	total: 1.77s	remaining: 8.22s
177:	learn: 0.4130590	total: 1.78s	remaining: 8.2s
178:	learn: 0.4129328	total: 1.79s	remaining: 8.2s

179:	learn: 0.4128289	total: 1.8s	remaining: 8.19s
180:	learn: 0.4127793	total: 1.81s	remaining: 8.18s
181:	learn: 0.4127028	total: 1.82s	remaining: 8.17s
182:	learn: 0.4125521	total: 1.83s	remaining: 8.16s
183:	learn: 0.4124283	total: 1.84s	remaining: 8.15s
184:	learn: 0.4123236	total: 1.85s	remaining: 8.14s
185:	learn: 0.4122889	total: 1.86s	remaining: 8.13s
186:	learn: 0.4121747	total: 1.87s	remaining: 8.12s
187:	learn: 0.4121253	total: 1.88s	remaining: 8.11s
188:	learn: 0.4120734	total: 1.89s	remaining: 8.09s
189:	learn: 0.4119805	total: 1.9s	remaining: 8.09s
190:	learn: 0.4118948	total: 1.91s	remaining: 8.08s
191:	learn: 0.4117866	total: 1.92s	remaining: 8.06s
192:	learn: 0.4117276	total: 1.93s	remaining: 8.05s
193:	learn: 0.4116597	total: 1.93s	remaining: 8.04s
194:	learn: 0.4115867	total: 1.94s	remaining: 8.02s
195:	learn: 0.4114235	total: 1.95s	remaining: 8.01s
196:	learn: 0.4112966	total: 1.96s	remaining: 8s
197:	learn: 0.4111552	total: 1.97s	remaining: 7.99s
198:	learn: 0.4110737	total: 1.98s	remaining: 7.98s
199:	learn: 0.4109512	total: 1.99s	remaining: 7.97s
200:	learn: 0.4108394	total: 2s	remaining: 7.96s
201:	learn: 0.4107752	total: 2.01s	remaining: 7.95s
202:	learn: 0.4106908	total: 2.02s	remaining: 7.94s
203:	learn: 0.4106422	total: 2.03s	remaining: 7.92s
204:	learn: 0.4106072	total: 2.04s	remaining: 7.91s
205:	learn: 0.4104907	total: 2.05s	remaining: 7.9s
206:	learn: 0.4103565	total: 2.06s	remaining: 7.89s
207:	learn: 0.4102632	total: 2.07s	remaining: 7.88s
208:	learn: 0.4101479	total: 2.08s	remaining: 7.87s
209:	learn: 0.4100241	total: 2.09s	remaining: 7.86s
210:	learn: 0.4099184	total: 2.1s	remaining: 7.85s
211:	learn: 0.4098570	total: 2.11s	remaining: 7.83s
212:	learn: 0.4097195	total: 2.12s	remaining: 7.82s
213:	learn: 0.4096430	total: 2.13s	remaining: 7.81s
214:	learn: 0.4095519	total: 2.14s	remaining: 7.8s
215:	learn: 0.4094012	total: 2.15s	remaining: 7.79s
216:	learn: 0.4093245	total: 2.16s	remaining: 7.78s
217:	learn: 0.4092583	total: 2.17s	remaining: 7.77s
218:	learn: 0.4091973	total: 2.17s	remaining: 7.76s
219:	learn: 0.4090690	total: 2.19s	remaining: 7.75s
220:	learn: 0.4089753	total: 2.19s	remaining: 7.74s
221:	learn: 0.4089387	total: 2.2s	remaining: 7.73s
222:	learn: 0.4088297	total: 2.21s	remaining: 7.72s
223:	learn: 0.4087580	total: 2.22s	remaining: 7.71s
224:	learn: 0.4086552	total: 2.23s	remaining: 7.7s
225:	learn: 0.4085914	total: 2.24s	remaining: 7.68s
226:	learn: 0.4084878	total: 2.25s	remaining: 7.67s
227:	learn: 0.4084287	total: 2.26s	remaining: 7.66s
228:	learn: 0.4082880	total: 2.27s	remaining: 7.65s
229:	learn: 0.4082016	total: 2.28s	remaining: 7.64s
230:	learn: 0.4080357	total: 2.29s	remaining: 7.63s
231:	learn: 0.4079489	total: 2.3s	remaining: 7.63s
232:	learn: 0.4079034	total: 2.31s	remaining: 7.62s
233:	learn: 0.4078215	total: 2.32s	remaining: 7.61s
234:	learn: 0.4077048	total: 2.33s	remaining: 7.6s
235:	learn: 0.4076191	total: 2.34s	remaining: 7.59s
236:	learn: 0.4075138	total: 2.35s	remaining: 7.58s
237:	learn: 0.4074122	total: 2.36s	remaining: 7.56s
238:	learn: 0.4073479	total: 2.37s	remaining: 7.55s

239:	learn: 0.4072279	total: 2.38s	remaining: 7.54s
240:	learn: 0.4071597	total: 2.39s	remaining: 7.53s
241:	learn: 0.4070361	total: 2.4s	remaining: 7.52s
242:	learn: 0.4069383	total: 2.41s	remaining: 7.51s
243:	learn: 0.4068429	total: 2.42s	remaining: 7.5s
244:	learn: 0.4067334	total: 2.43s	remaining: 7.49s
245:	learn: 0.4066294	total: 2.44s	remaining: 7.48s
246:	learn: 0.4065081	total: 2.45s	remaining: 7.47s
247:	learn: 0.4064398	total: 2.46s	remaining: 7.46s
248:	learn: 0.4062922	total: 2.47s	remaining: 7.45s
249:	learn: 0.4061898	total: 2.48s	remaining: 7.44s
250:	learn: 0.4060594	total: 2.49s	remaining: 7.43s
251:	learn: 0.4059444	total: 2.5s	remaining: 7.42s
252:	learn: 0.4058446	total: 2.51s	remaining: 7.41s
253:	learn: 0.4057387	total: 2.52s	remaining: 7.4s
254:	learn: 0.4056967	total: 2.53s	remaining: 7.39s
255:	learn: 0.4055862	total: 2.54s	remaining: 7.38s
256:	learn: 0.4055080	total: 2.55s	remaining: 7.37s
257:	learn: 0.4054533	total: 2.56s	remaining: 7.35s
258:	learn: 0.4053795	total: 2.57s	remaining: 7.34s
259:	learn: 0.4053227	total: 2.58s	remaining: 7.33s
260:	learn: 0.4052208	total: 2.58s	remaining: 7.32s
261:	learn: 0.4051152	total: 2.59s	remaining: 7.31s
262:	learn: 0.4050160	total: 2.6s	remaining: 7.3s
263:	learn: 0.4048732	total: 2.61s	remaining: 7.29s
264:	learn: 0.4047959	total: 2.62s	remaining: 7.27s
265:	learn: 0.4046877	total: 2.63s	remaining: 7.26s
266:	learn: 0.4045634	total: 2.64s	remaining: 7.25s
267:	learn: 0.4044216	total: 2.65s	remaining: 7.24s
268:	learn: 0.4043584	total: 2.66s	remaining: 7.23s
269:	learn: 0.4043157	total: 2.67s	remaining: 7.22s
270:	learn: 0.4042326	total: 2.68s	remaining: 7.21s
271:	learn: 0.4041515	total: 2.69s	remaining: 7.19s
272:	learn: 0.4040685	total: 2.7s	remaining: 7.18s
273:	learn: 0.4039031	total: 2.71s	remaining: 7.17s
274:	learn: 0.4037676	total: 2.72s	remaining: 7.16s
275:	learn: 0.4036763	total: 2.73s	remaining: 7.15s
276:	learn: 0.4035174	total: 2.73s	remaining: 7.14s
277:	learn: 0.4033628	total: 2.75s	remaining: 7.13s
278:	learn: 0.4032023	total: 2.75s	remaining: 7.12s
279:	learn: 0.4031849	total: 2.76s	remaining: 7.11s
280:	learn: 0.4029786	total: 2.77s	remaining: 7.1s
281:	learn: 0.4028879	total: 2.78s	remaining: 7.09s
282:	learn: 0.4027524	total: 2.79s	remaining: 7.08s
283:	learn: 0.4026421	total: 2.8s	remaining: 7.07s
284:	learn: 0.4025434	total: 2.81s	remaining: 7.05s
285:	learn: 0.4024418	total: 2.82s	remaining: 7.04s
286:	learn: 0.4023723	total: 2.83s	remaining: 7.03s
287:	learn: 0.4022639	total: 2.84s	remaining: 7.02s
288:	learn: 0.4021819	total: 2.85s	remaining: 7.01s
289:	learn: 0.4020314	total: 2.86s	remaining: 7s
290:	learn: 0.4019383	total: 2.87s	remaining: 6.99s
291:	learn: 0.4017892	total: 2.88s	remaining: 6.98s
292:	learn: 0.4017072	total: 2.89s	remaining: 6.96s
293:	learn: 0.4016391	total: 2.9s	remaining: 6.95s
294:	learn: 0.4015599	total: 2.9s	remaining: 6.94s
295:	learn: 0.4014399	total: 2.92s	remaining: 6.93s
296:	learn: 0.4013340	total: 2.92s	remaining: 6.92s
297:	learn: 0.4012051	total: 2.93s	remaining: 6.91s
298:	learn: 0.4011354	total: 2.94s	remaining: 6.9s

299:	learn: 0.4010005	total: 2.95s	remaining: 6.89s
300:	learn: 0.4008222	total: 2.96s	remaining: 6.88s
301:	learn: 0.4006931	total: 2.98s	remaining: 6.88s
302:	learn: 0.4005716	total: 2.98s	remaining: 6.87s
303:	learn: 0.4004318	total: 3s	remaining: 6.86s
304:	learn: 0.4003303	total: 3s	remaining: 6.85s
305:	learn: 0.4001977	total: 3.01s	remaining: 6.83s
306:	learn: 0.4000795	total: 3.02s	remaining: 6.83s
307:	learn: 0.4000126	total: 3.03s	remaining: 6.82s
308:	learn: 0.3999304	total: 3.04s	remaining: 6.8s
309:	learn: 0.3998124	total: 3.05s	remaining: 6.79s
310:	learn: 0.3997282	total: 3.06s	remaining: 6.78s
311:	learn: 0.3996855	total: 3.07s	remaining: 6.77s
312:	learn: 0.3995509	total: 3.08s	remaining: 6.76s
313:	learn: 0.3993837	total: 3.09s	remaining: 6.75s
314:	learn: 0.3993087	total: 3.1s	remaining: 6.75s
315:	learn: 0.3992149	total: 3.11s	remaining: 6.74s
316:	learn: 0.3991411	total: 3.12s	remaining: 6.73s
317:	learn: 0.3990356	total: 3.13s	remaining: 6.72s
318:	learn: 0.3989649	total: 3.14s	remaining: 6.71s
319:	learn: 0.3988016	total: 3.15s	remaining: 6.7s
320:	learn: 0.3986840	total: 3.16s	remaining: 6.69s
321:	learn: 0.3985382	total: 3.17s	remaining: 6.68s
322:	learn: 0.3984596	total: 3.18s	remaining: 6.67s
323:	learn: 0.3983495	total: 3.19s	remaining: 6.66s
324:	learn: 0.3982280	total: 3.2s	remaining: 6.65s
325:	learn: 0.3980498	total: 3.21s	remaining: 6.64s
326:	learn: 0.3979686	total: 3.22s	remaining: 6.63s
327:	learn: 0.3978100	total: 3.23s	remaining: 6.62s
328:	learn: 0.3977544	total: 3.24s	remaining: 6.61s
329:	learn: 0.3976518	total: 3.25s	remaining: 6.6s
330:	learn: 0.3975752	total: 3.26s	remaining: 6.59s
331:	learn: 0.3974883	total: 3.27s	remaining: 6.58s
332:	learn: 0.3973785	total: 3.28s	remaining: 6.57s
333:	learn: 0.3972213	total: 3.29s	remaining: 6.57s
334:	learn: 0.3971147	total: 3.3s	remaining: 6.56s
335:	learn: 0.3969106	total: 3.31s	remaining: 6.55s
336:	learn: 0.3968028	total: 3.32s	remaining: 6.54s
337:	learn: 0.3967097	total: 3.33s	remaining: 6.53s
338:	learn: 0.3966346	total: 3.34s	remaining: 6.52s
339:	learn: 0.3965627	total: 3.35s	remaining: 6.5s
340:	learn: 0.3963965	total: 3.36s	remaining: 6.5s
341:	learn: 0.3963190	total: 3.37s	remaining: 6.49s
342:	learn: 0.3962078	total: 3.38s	remaining: 6.48s
343:	learn: 0.3960983	total: 3.39s	remaining: 6.46s
344:	learn: 0.3960075	total: 3.4s	remaining: 6.46s
345:	learn: 0.3959328	total: 3.41s	remaining: 6.44s
346:	learn: 0.3957981	total: 3.42s	remaining: 6.43s
347:	learn: 0.3956814	total: 3.43s	remaining: 6.42s
348:	learn: 0.3955602	total: 3.44s	remaining: 6.42s
349:	learn: 0.3954391	total: 3.45s	remaining: 6.41s
350:	learn: 0.3953127	total: 3.46s	remaining: 6.4s
351:	learn: 0.3951895	total: 3.47s	remaining: 6.39s
352:	learn: 0.3950923	total: 3.48s	remaining: 6.38s
353:	learn: 0.3950055	total: 3.49s	remaining: 6.37s
354:	learn: 0.3948981	total: 3.5s	remaining: 6.36s
355:	learn: 0.3948165	total: 3.51s	remaining: 6.35s
356:	learn: 0.3946942	total: 3.52s	remaining: 6.34s
357:	learn: 0.3945299	total: 3.53s	remaining: 6.33s
358:	learn: 0.3943895	total: 3.54s	remaining: 6.32s

359:	learn: 0.3942825	total: 3.55s	remaining: 6.31s
360:	learn: 0.3941741	total: 3.56s	remaining: 6.3s
361:	learn: 0.3940827	total: 3.57s	remaining: 6.29s
362:	learn: 0.3940030	total: 3.58s	remaining: 6.28s
363:	learn: 0.3938935	total: 3.59s	remaining: 6.27s
364:	learn: 0.3937160	total: 3.6s	remaining: 6.26s
365:	learn: 0.3936571	total: 3.61s	remaining: 6.25s
366:	learn: 0.3934956	total: 3.62s	remaining: 6.25s
367:	learn: 0.3933792	total: 3.63s	remaining: 6.24s
368:	learn: 0.3932901	total: 3.64s	remaining: 6.22s
369:	learn: 0.3932041	total: 3.65s	remaining: 6.21s
370:	learn: 0.3930799	total: 3.66s	remaining: 6.21s
371:	learn: 0.3929940	total: 3.67s	remaining: 6.2s
372:	learn: 0.3929100	total: 3.68s	remaining: 6.18s
373:	learn: 0.3927591	total: 3.69s	remaining: 6.17s
374:	learn: 0.3926735	total: 3.7s	remaining: 6.16s
375:	learn: 0.3925980	total: 3.71s	remaining: 6.15s
376:	learn: 0.3924954	total: 3.72s	remaining: 6.14s
377:	learn: 0.3923810	total: 3.73s	remaining: 6.13s
378:	learn: 0.3922674	total: 3.74s	remaining: 6.12s
379:	learn: 0.3921728	total: 3.75s	remaining: 6.11s
380:	learn: 0.3920643	total: 3.75s	remaining: 6.1s
381:	learn: 0.3919582	total: 3.76s	remaining: 6.09s
382:	learn: 0.3918785	total: 3.77s	remaining: 6.08s
383:	learn: 0.3917956	total: 3.78s	remaining: 6.07s
384:	learn: 0.3917009	total: 3.79s	remaining: 6.06s
385:	learn: 0.3916168	total: 3.8s	remaining: 6.05s
386:	learn: 0.3915174	total: 3.81s	remaining: 6.04s
387:	learn: 0.3914578	total: 3.82s	remaining: 6.03s
388:	learn: 0.3912968	total: 3.83s	remaining: 6.02s
389:	learn: 0.3911843	total: 3.84s	remaining: 6.01s
390:	learn: 0.3911412	total: 3.85s	remaining: 5.99s
391:	learn: 0.3910758	total: 3.86s	remaining: 5.98s
392:	learn: 0.3909468	total: 3.87s	remaining: 5.97s
393:	learn: 0.3908759	total: 3.88s	remaining: 5.96s
394:	learn: 0.3908230	total: 3.89s	remaining: 5.95s
395:	learn: 0.3907390	total: 3.9s	remaining: 5.94s
396:	learn: 0.3906335	total: 3.9s	remaining: 5.93s
397:	learn: 0.3904870	total: 3.91s	remaining: 5.92s
398:	learn: 0.3903690	total: 3.92s	remaining: 5.91s
399:	learn: 0.3902530	total: 3.93s	remaining: 5.9s
400:	learn: 0.3901545	total: 3.94s	remaining: 5.89s
401:	learn: 0.3900462	total: 3.95s	remaining: 5.88s
402:	learn: 0.3899412	total: 3.96s	remaining: 5.87s
403:	learn: 0.3898652	total: 3.97s	remaining: 5.86s
404:	learn: 0.3897575	total: 3.98s	remaining: 5.85s
405:	learn: 0.3896414	total: 3.99s	remaining: 5.84s
406:	learn: 0.3895568	total: 4s	remaining: 5.83s
407:	learn: 0.3894405	total: 4.01s	remaining: 5.82s
408:	learn: 0.3892718	total: 4.02s	remaining: 5.81s
409:	learn: 0.3891584	total: 4.03s	remaining: 5.8s
410:	learn: 0.3890607	total: 4.04s	remaining: 5.79s
411:	learn: 0.3889347	total: 4.05s	remaining: 5.78s
412:	learn: 0.3888571	total: 4.06s	remaining: 5.77s
413:	learn: 0.3887367	total: 4.07s	remaining: 5.76s
414:	learn: 0.3886960	total: 4.08s	remaining: 5.75s
415:	learn: 0.3885623	total: 4.09s	remaining: 5.74s
416:	learn: 0.3884231	total: 4.1s	remaining: 5.73s
417:	learn: 0.3882223	total: 4.11s	remaining: 5.72s
418:	learn: 0.3880751	total: 4.12s	remaining: 5.71s

419:	learn: 0.3879545	total: 4.13s	remaining: 5.7s
420:	learn: 0.3879041	total: 4.14s	remaining: 5.69s
421:	learn: 0.3878047	total: 4.15s	remaining: 5.68s
422:	learn: 0.3876894	total: 4.16s	remaining: 5.67s
423:	learn: 0.3875577	total: 4.17s	remaining: 5.66s
424:	learn: 0.3874196	total: 4.18s	remaining: 5.65s
425:	learn: 0.3872762	total: 4.19s	remaining: 5.64s
426:	learn: 0.3871580	total: 4.2s	remaining: 5.63s
427:	learn: 0.3871074	total: 4.21s	remaining: 5.62s
428:	learn: 0.3870107	total: 4.21s	remaining: 5.61s
429:	learn: 0.3869253	total: 4.22s	remaining: 5.6s
430:	learn: 0.3868458	total: 4.23s	remaining: 5.59s
431:	learn: 0.3868115	total: 4.24s	remaining: 5.58s
432:	learn: 0.3866946	total: 4.25s	remaining: 5.57s
433:	learn: 0.3866227	total: 4.26s	remaining: 5.56s
434:	learn: 0.3864754	total: 4.27s	remaining: 5.55s
435:	learn: 0.3863990	total: 4.28s	remaining: 5.54s
436:	learn: 0.3863275	total: 4.29s	remaining: 5.53s
437:	learn: 0.3862631	total: 4.3s	remaining: 5.52s
438:	learn: 0.3861745	total: 4.31s	remaining: 5.51s
439:	learn: 0.3860262	total: 4.32s	remaining: 5.5s
440:	learn: 0.3859228	total: 4.33s	remaining: 5.49s
441:	learn: 0.3858101	total: 4.34s	remaining: 5.48s
442:	learn: 0.3856890	total: 4.35s	remaining: 5.47s
443:	learn: 0.3855330	total: 4.36s	remaining: 5.46s
444:	learn: 0.3854471	total: 4.37s	remaining: 5.45s
445:	learn: 0.3853854	total: 4.38s	remaining: 5.44s
446:	learn: 0.3853218	total: 4.39s	remaining: 5.43s
447:	learn: 0.3852505	total: 4.4s	remaining: 5.42s
448:	learn: 0.3851451	total: 4.41s	remaining: 5.41s
449:	learn: 0.3850272	total: 4.42s	remaining: 5.4s
450:	learn: 0.3849297	total: 4.43s	remaining: 5.39s
451:	learn: 0.3848901	total: 4.44s	remaining: 5.38s
452:	learn: 0.3847951	total: 4.45s	remaining: 5.37s
453:	learn: 0.3847282	total: 4.46s	remaining: 5.36s
454:	learn: 0.3846371	total: 4.46s	remaining: 5.35s
455:	learn: 0.3845063	total: 4.47s	remaining: 5.34s
456:	learn: 0.3844284	total: 4.48s	remaining: 5.33s
457:	learn: 0.3843593	total: 4.49s	remaining: 5.32s
458:	learn: 0.3842323	total: 4.5s	remaining: 5.31s
459:	learn: 0.3841158	total: 4.51s	remaining: 5.3s
460:	learn: 0.3839704	total: 4.52s	remaining: 5.29s
461:	learn: 0.3838654	total: 4.53s	remaining: 5.28s
462:	learn: 0.3838026	total: 4.54s	remaining: 5.27s
463:	learn: 0.3836216	total: 4.55s	remaining: 5.26s
464:	learn: 0.3835142	total: 4.56s	remaining: 5.25s
465:	learn: 0.3834349	total: 4.58s	remaining: 5.24s
466:	learn: 0.3833725	total: 4.58s	remaining: 5.23s
467:	learn: 0.3832752	total: 4.59s	remaining: 5.22s
468:	learn: 0.3831665	total: 4.61s	remaining: 5.21s
469:	learn: 0.3830653	total: 4.62s	remaining: 5.2s
470:	learn: 0.3829460	total: 4.63s	remaining: 5.2s
471:	learn: 0.3828700	total: 4.63s	remaining: 5.18s
472:	learn: 0.3827759	total: 4.64s	remaining: 5.17s
473:	learn: 0.3826495	total: 4.65s	remaining: 5.17s
474:	learn: 0.3825210	total: 4.66s	remaining: 5.16s
475:	learn: 0.3824086	total: 4.67s	remaining: 5.15s
476:	learn: 0.3823369	total: 4.68s	remaining: 5.13s
477:	learn: 0.3822092	total: 4.69s	remaining: 5.13s
478:	learn: 0.3821073	total: 4.7s	remaining: 5.12s

479:	learn: 0.3819709	total: 4.71s	remaining: 5.11s
480:	learn: 0.3818520	total: 4.72s	remaining: 5.09s
481:	learn: 0.3817795	total: 4.73s	remaining: 5.08s
482:	learn: 0.3816794	total: 4.74s	remaining: 5.08s
483:	learn: 0.3816092	total: 4.75s	remaining: 5.07s
484:	learn: 0.3814980	total: 4.76s	remaining: 5.05s
485:	learn: 0.3814472	total: 4.77s	remaining: 5.04s
486:	learn: 0.3813775	total: 4.78s	remaining: 5.03s
487:	learn: 0.3813398	total: 4.79s	remaining: 5.02s
488:	learn: 0.3812736	total: 4.8s	remaining: 5.01s
489:	learn: 0.3811510	total: 4.81s	remaining: 5s
490:	learn: 0.3810643	total: 4.82s	remaining: 4.99s
491:	learn: 0.3809273	total: 4.83s	remaining: 4.99s
492:	learn: 0.3808024	total: 4.84s	remaining: 4.98s
493:	learn: 0.3806999	total: 4.85s	remaining: 4.97s
494:	learn: 0.3806301	total: 4.86s	remaining: 4.96s
495:	learn: 0.3805589	total: 4.87s	remaining: 4.95s
496:	learn: 0.3804885	total: 4.88s	remaining: 4.94s
497:	learn: 0.3804302	total: 4.89s	remaining: 4.93s
498:	learn: 0.3802965	total: 4.9s	remaining: 4.92s
499:	learn: 0.3802225	total: 4.91s	remaining: 4.91s
500:	learn: 0.3800774	total: 4.92s	remaining: 4.9s
501:	learn: 0.3800262	total: 4.93s	remaining: 4.89s
502:	learn: 0.3799594	total: 4.94s	remaining: 4.88s
503:	learn: 0.3798464	total: 4.95s	remaining: 4.87s
504:	learn: 0.3797890	total: 4.96s	remaining: 4.86s
505:	learn: 0.3797147	total: 4.97s	remaining: 4.85s
506:	learn: 0.3796208	total: 4.98s	remaining: 4.84s
507:	learn: 0.3795414	total: 4.99s	remaining: 4.83s
508:	learn: 0.3794768	total: 4.99s	remaining: 4.82s
509:	learn: 0.3794187	total: 5s	remaining: 4.81s
510:	learn: 0.3793403	total: 5.01s	remaining: 4.8s
511:	learn: 0.3792761	total: 5.02s	remaining: 4.79s
512:	learn: 0.3792084	total: 5.03s	remaining: 4.78s
513:	learn: 0.3791033	total: 5.04s	remaining: 4.77s
514:	learn: 0.3790090	total: 5.05s	remaining: 4.76s
515:	learn: 0.3789670	total: 5.06s	remaining: 4.75s
516:	learn: 0.3789472	total: 5.07s	remaining: 4.74s
517:	learn: 0.3788033	total: 5.08s	remaining: 4.73s
518:	learn: 0.3787650	total: 5.09s	remaining: 4.72s
519:	learn: 0.3786896	total: 5.1s	remaining: 4.71s
520:	learn: 0.3786414	total: 5.11s	remaining: 4.7s
521:	learn: 0.3785800	total: 5.12s	remaining: 4.69s
522:	learn: 0.3785034	total: 5.13s	remaining: 4.68s
523:	learn: 0.3784260	total: 5.14s	remaining: 4.67s
524:	learn: 0.3783495	total: 5.14s	remaining: 4.66s
525:	learn: 0.3782310	total: 5.16s	remaining: 4.64s
526:	learn: 0.3781262	total: 5.16s	remaining: 4.63s
527:	learn: 0.3780384	total: 5.17s	remaining: 4.63s
528:	learn: 0.3779768	total: 5.18s	remaining: 4.62s
529:	learn: 0.3779305	total: 5.19s	remaining: 4.61s
530:	learn: 0.3778259	total: 5.2s	remaining: 4.59s
531:	learn: 0.3777651	total: 5.21s	remaining: 4.58s
532:	learn: 0.3776600	total: 5.22s	remaining: 4.57s
533:	learn: 0.3775741	total: 5.23s	remaining: 4.56s
534:	learn: 0.3774477	total: 5.24s	remaining: 4.55s
535:	learn: 0.3773286	total: 5.25s	remaining: 4.54s
536:	learn: 0.3772192	total: 5.26s	remaining: 4.54s
537:	learn: 0.3771390	total: 5.27s	remaining: 4.53s
538:	learn: 0.3770582	total: 5.28s	remaining: 4.51s

539:	learn: 0.3770199	total: 5.29s	remaining: 4.5s
540:	learn: 0.3768541	total: 5.3s	remaining: 4.5s
541:	learn: 0.3768127	total: 5.31s	remaining: 4.48s
542:	learn: 0.3767452	total: 5.32s	remaining: 4.47s
543:	learn: 0.3766495	total: 5.33s	remaining: 4.46s
544:	learn: 0.3765462	total: 5.33s	remaining: 4.45s
545:	learn: 0.3764280	total: 5.34s	remaining: 4.44s
546:	learn: 0.3763469	total: 5.36s	remaining: 4.43s
547:	learn: 0.3763024	total: 5.36s	remaining: 4.42s
548:	learn: 0.3761894	total: 5.37s	remaining: 4.41s
549:	learn: 0.3761175	total: 5.38s	remaining: 4.4s
550:	learn: 0.3760022	total: 5.39s	remaining: 4.39s
551:	learn: 0.3759260	total: 5.4s	remaining: 4.38s
552:	learn: 0.3758233	total: 5.41s	remaining: 4.38s
553:	learn: 0.3757558	total: 5.42s	remaining: 4.37s
554:	learn: 0.3756683	total: 5.43s	remaining: 4.36s
555:	learn: 0.3756076	total: 5.44s	remaining: 4.34s
556:	learn: 0.3755396	total: 5.45s	remaining: 4.33s
557:	learn: 0.3754492	total: 5.46s	remaining: 4.33s
558:	learn: 0.3753434	total: 5.47s	remaining: 4.32s
559:	learn: 0.3753114	total: 5.48s	remaining: 4.3s
560:	learn: 0.3751858	total: 5.49s	remaining: 4.29s
561:	learn: 0.3751215	total: 5.5s	remaining: 4.29s
562:	learn: 0.3750313	total: 5.51s	remaining: 4.28s
563:	learn: 0.3749672	total: 5.52s	remaining: 4.27s
564:	learn: 0.3748653	total: 5.53s	remaining: 4.26s
565:	learn: 0.3748212	total: 5.54s	remaining: 4.25s
566:	learn: 0.3747561	total: 5.55s	remaining: 4.24s
567:	learn: 0.3746642	total: 5.56s	remaining: 4.23s
568:	learn: 0.3745777	total: 5.57s	remaining: 4.22s
569:	learn: 0.3745232	total: 5.58s	remaining: 4.21s
570:	learn: 0.3744613	total: 5.59s	remaining: 4.2s
571:	learn: 0.3743761	total: 5.6s	remaining: 4.19s
572:	learn: 0.3742967	total: 5.61s	remaining: 4.18s
573:	learn: 0.3742380	total: 5.62s	remaining: 4.17s
574:	learn: 0.3740983	total: 5.63s	remaining: 4.16s
575:	learn: 0.3740276	total: 5.64s	remaining: 4.15s
576:	learn: 0.3738909	total: 5.65s	remaining: 4.14s
577:	learn: 0.3738283	total: 5.66s	remaining: 4.13s
578:	learn: 0.3737602	total: 5.67s	remaining: 4.12s
579:	learn: 0.3736285	total: 5.68s	remaining: 4.11s
580:	learn: 0.3734853	total: 5.69s	remaining: 4.1s
581:	learn: 0.3734206	total: 5.7s	remaining: 4.09s
582:	learn: 0.3732636	total: 5.71s	remaining: 4.08s
583:	learn: 0.3731541	total: 5.72s	remaining: 4.07s
584:	learn: 0.3730452	total: 5.73s	remaining: 4.06s
585:	learn: 0.3730061	total: 5.74s	remaining: 4.05s
586:	learn: 0.3729084	total: 5.75s	remaining: 4.04s
587:	learn: 0.3728514	total: 5.76s	remaining: 4.03s
588:	learn: 0.3727854	total: 5.77s	remaining: 4.02s
589:	learn: 0.3726701	total: 5.78s	remaining: 4.01s
590:	learn: 0.3726097	total: 5.79s	remaining: 4s
591:	learn: 0.3725285	total: 5.79s	remaining: 3.99s
592:	learn: 0.3724364	total: 5.8s	remaining: 3.98s
593:	learn: 0.3723793	total: 5.81s	remaining: 3.97s
594:	learn: 0.3722827	total: 5.82s	remaining: 3.96s
595:	learn: 0.3722406	total: 5.83s	remaining: 3.95s
596:	learn: 0.3721189	total: 5.84s	remaining: 3.94s
597:	learn: 0.3719995	total: 5.85s	remaining: 3.94s
598:	learn: 0.3718946	total: 5.86s	remaining: 3.92s

599:	learn: 0.3718191	total: 5.87s	remaining: 3.92s
600:	learn: 0.3717224	total: 5.88s	remaining: 3.9s
601:	learn: 0.3716700	total: 5.89s	remaining: 3.89s
602:	learn: 0.3715886	total: 5.9s	remaining: 3.88s
603:	learn: 0.3714740	total: 5.91s	remaining: 3.87s
604:	learn: 0.3713582	total: 5.92s	remaining: 3.87s
605:	learn: 0.3712535	total: 5.93s	remaining: 3.85s
606:	learn: 0.3711173	total: 5.94s	remaining: 3.85s
607:	learn: 0.3710250	total: 5.95s	remaining: 3.83s
608:	learn: 0.3709068	total: 5.96s	remaining: 3.83s
609:	learn: 0.3708827	total: 5.97s	remaining: 3.81s
610:	learn: 0.3707868	total: 5.98s	remaining: 3.81s
611:	learn: 0.3707047	total: 5.99s	remaining: 3.79s
612:	learn: 0.3705954	total: 6s	remaining: 3.79s
613:	learn: 0.3704739	total: 6s	remaining: 3.77s
614:	learn: 0.3703626	total: 6.01s	remaining: 3.77s
615:	learn: 0.3702736	total: 6.03s	remaining: 3.76s
616:	learn: 0.3701845	total: 6.03s	remaining: 3.75s
617:	learn: 0.3701076	total: 6.04s	remaining: 3.74s
618:	learn: 0.3700517	total: 6.05s	remaining: 3.73s
619:	learn: 0.3699293	total: 6.06s	remaining: 3.72s
620:	learn: 0.3698429	total: 6.07s	remaining: 3.71s
621:	learn: 0.3697666	total: 6.08s	remaining: 3.7s
622:	learn: 0.3696950	total: 6.09s	remaining: 3.69s
623:	learn: 0.3696603	total: 6.1s	remaining: 3.68s
624:	learn: 0.3695782	total: 6.11s	remaining: 3.67s
625:	learn: 0.3694982	total: 6.12s	remaining: 3.66s
626:	learn: 0.3694273	total: 6.13s	remaining: 3.65s
627:	learn: 0.3693410	total: 6.14s	remaining: 3.64s
628:	learn: 0.3692458	total: 6.15s	remaining: 3.63s
629:	learn: 0.3691022	total: 6.16s	remaining: 3.62s
630:	learn: 0.3690222	total: 6.17s	remaining: 3.61s
631:	learn: 0.3689897	total: 6.18s	remaining: 3.6s
632:	learn: 0.3689315	total: 6.19s	remaining: 3.59s
633:	learn: 0.3688988	total: 6.2s	remaining: 3.58s
634:	learn: 0.3688058	total: 6.21s	remaining: 3.57s
635:	learn: 0.3687133	total: 6.22s	remaining: 3.56s
636:	learn: 0.3685993	total: 6.23s	remaining: 3.55s
637:	learn: 0.3685322	total: 6.24s	remaining: 3.54s
638:	learn: 0.3684609	total: 6.25s	remaining: 3.53s
639:	learn: 0.3683343	total: 6.25s	remaining: 3.52s
640:	learn: 0.3682804	total: 6.26s	remaining: 3.51s
641:	learn: 0.3681624	total: 6.27s	remaining: 3.5s
642:	learn: 0.3680980	total: 6.28s	remaining: 3.49s
643:	learn: 0.3680402	total: 6.29s	remaining: 3.48s
644:	learn: 0.3680055	total: 6.3s	remaining: 3.47s
645:	learn: 0.3679135	total: 6.31s	remaining: 3.46s
646:	learn: 0.3678236	total: 6.32s	remaining: 3.45s
647:	learn: 0.3677224	total: 6.33s	remaining: 3.44s
648:	learn: 0.3676400	total: 6.34s	remaining: 3.43s
649:	learn: 0.3675821	total: 6.35s	remaining: 3.42s
650:	learn: 0.3674863	total: 6.36s	remaining: 3.41s
651:	learn: 0.3674220	total: 6.37s	remaining: 3.4s
652:	learn: 0.3673281	total: 6.38s	remaining: 3.39s
653:	learn: 0.3672770	total: 6.39s	remaining: 3.38s
654:	learn: 0.3671927	total: 6.4s	remaining: 3.37s
655:	learn: 0.3671136	total: 6.41s	remaining: 3.36s
656:	learn: 0.3670368	total: 6.42s	remaining: 3.35s
657:	learn: 0.3669433	total: 6.42s	remaining: 3.34s
658:	learn: 0.3669210	total: 6.43s	remaining: 3.33s

659:	learn: 0.3668065	total: 6.44s	remaining: 3.32s
660:	learn: 0.3666755	total: 6.45s	remaining: 3.31s
661:	learn: 0.3665711	total: 6.46s	remaining: 3.3s
662:	learn: 0.3664163	total: 6.47s	remaining: 3.29s
663:	learn: 0.3662748	total: 6.48s	remaining: 3.28s
664:	learn: 0.3661879	total: 6.49s	remaining: 3.27s
665:	learn: 0.3661589	total: 6.5s	remaining: 3.26s
666:	learn: 0.3660819	total: 6.51s	remaining: 3.25s
667:	learn: 0.3660213	total: 6.52s	remaining: 3.24s
668:	learn: 0.3659355	total: 6.53s	remaining: 3.23s
669:	learn: 0.3658836	total: 6.54s	remaining: 3.22s
670:	learn: 0.3657675	total: 6.55s	remaining: 3.21s
671:	learn: 0.3656894	total: 6.56s	remaining: 3.2s
672:	learn: 0.3655981	total: 6.57s	remaining: 3.19s
673:	learn: 0.3655156	total: 6.58s	remaining: 3.18s
674:	learn: 0.3654278	total: 6.59s	remaining: 3.17s
675:	learn: 0.3653163	total: 6.6s	remaining: 3.16s
676:	learn: 0.3652398	total: 6.61s	remaining: 3.15s
677:	learn: 0.3651543	total: 6.62s	remaining: 3.15s
678:	learn: 0.3650925	total: 6.63s	remaining: 3.13s
679:	learn: 0.3650271	total: 6.64s	remaining: 3.13s
680:	learn: 0.3649787	total: 6.65s	remaining: 3.12s
681:	learn: 0.3648464	total: 6.66s	remaining: 3.11s
682:	learn: 0.3647160	total: 6.67s	remaining: 3.1s
683:	learn: 0.3646068	total: 6.68s	remaining: 3.09s
684:	learn: 0.3644442	total: 6.69s	remaining: 3.08s
685:	learn: 0.3643420	total: 6.7s	remaining: 3.07s
686:	learn: 0.3642700	total: 6.71s	remaining: 3.06s
687:	learn: 0.3642286	total: 6.72s	remaining: 3.05s
688:	learn: 0.3641422	total: 6.73s	remaining: 3.04s
689:	learn: 0.3641092	total: 6.74s	remaining: 3.03s
690:	learn: 0.3639942	total: 6.75s	remaining: 3.02s
691:	learn: 0.3639186	total: 6.76s	remaining: 3.01s
692:	learn: 0.3638768	total: 6.77s	remaining: 3s
693:	learn: 0.3638109	total: 6.78s	remaining: 2.99s
694:	learn: 0.3636933	total: 6.79s	remaining: 2.98s
695:	learn: 0.3636573	total: 6.8s	remaining: 2.97s
696:	learn: 0.3635448	total: 6.81s	remaining: 2.96s
697:	learn: 0.3634582	total: 6.82s	remaining: 2.95s
698:	learn: 0.3633950	total: 6.83s	remaining: 2.94s
699:	learn: 0.3633293	total: 6.84s	remaining: 2.93s
700:	learn: 0.3632271	total: 6.85s	remaining: 2.92s
701:	learn: 0.3631837	total: 6.86s	remaining: 2.91s
702:	learn: 0.3630871	total: 6.87s	remaining: 2.9s
703:	learn: 0.3629995	total: 6.88s	remaining: 2.89s
704:	learn: 0.3628857	total: 6.89s	remaining: 2.88s
705:	learn: 0.3628055	total: 6.9s	remaining: 2.87s
706:	learn: 0.3627102	total: 6.91s	remaining: 2.86s
707:	learn: 0.3626181	total: 6.92s	remaining: 2.85s
708:	learn: 0.3625753	total: 6.93s	remaining: 2.84s
709:	learn: 0.3624968	total: 6.93s	remaining: 2.83s
710:	learn: 0.3623796	total: 6.95s	remaining: 2.82s
711:	learn: 0.3623361	total: 6.95s	remaining: 2.81s
712:	learn: 0.3622363	total: 6.96s	remaining: 2.8s
713:	learn: 0.3621525	total: 6.97s	remaining: 2.79s
714:	learn: 0.3620692	total: 6.98s	remaining: 2.78s
715:	learn: 0.3620189	total: 6.99s	remaining: 2.77s
716:	learn: 0.3619407	total: 7s	remaining: 2.76s
717:	learn: 0.3618728	total: 7.01s	remaining: 2.75s
718:	learn: 0.3617710	total: 7.02s	remaining: 2.75s

719:	learn: 0.3616145	total: 7.03s	remaining: 2.73s
720:	learn: 0.3615128	total: 7.04s	remaining: 2.73s
721:	learn: 0.3614647	total: 7.05s	remaining: 2.71s
722:	learn: 0.3613819	total: 7.06s	remaining: 2.71s
723:	learn: 0.3612741	total: 7.07s	remaining: 2.7s
724:	learn: 0.3611837	total: 7.08s	remaining: 2.69s
725:	learn: 0.3611201	total: 7.09s	remaining: 2.68s
726:	learn: 0.3610670	total: 7.1s	remaining: 2.67s
727:	learn: 0.3609124	total: 7.11s	remaining: 2.66s
728:	learn: 0.3608248	total: 7.12s	remaining: 2.65s
729:	learn: 0.3607142	total: 7.13s	remaining: 2.64s
730:	learn: 0.3605838	total: 7.14s	remaining: 2.63s
731:	learn: 0.3605265	total: 7.15s	remaining: 2.62s
732:	learn: 0.3604387	total: 7.16s	remaining: 2.61s
733:	learn: 0.3604003	total: 7.17s	remaining: 2.6s
734:	learn: 0.3603151	total: 7.18s	remaining: 2.59s
735:	learn: 0.3602578	total: 7.19s	remaining: 2.58s
736:	learn: 0.3602196	total: 7.2s	remaining: 2.57s
737:	learn: 0.3602184	total: 7.21s	remaining: 2.56s
738:	learn: 0.3601231	total: 7.22s	remaining: 2.55s
739:	learn: 0.3600445	total: 7.23s	remaining: 2.54s
740:	learn: 0.3599774	total: 7.24s	remaining: 2.53s
741:	learn: 0.3599215	total: 7.25s	remaining: 2.52s
742:	learn: 0.3598011	total: 7.26s	remaining: 2.51s
743:	learn: 0.3597216	total: 7.27s	remaining: 2.5s
744:	learn: 0.3596467	total: 7.28s	remaining: 2.49s
745:	learn: 0.3596327	total: 7.29s	remaining: 2.48s
746:	learn: 0.3595135	total: 7.3s	remaining: 2.47s
747:	learn: 0.3594620	total: 7.31s	remaining: 2.46s
748:	learn: 0.3593580	total: 7.32s	remaining: 2.45s
749:	learn: 0.3593244	total: 7.33s	remaining: 2.44s
750:	learn: 0.3592096	total: 7.34s	remaining: 2.43s
751:	learn: 0.3591082	total: 7.35s	remaining: 2.42s
752:	learn: 0.3590753	total: 7.36s	remaining: 2.41s
753:	learn: 0.3589922	total: 7.37s	remaining: 2.4s
754:	learn: 0.3589103	total: 7.38s	remaining: 2.39s
755:	learn: 0.3588437	total: 7.39s	remaining: 2.38s
756:	learn: 0.3587795	total: 7.39s	remaining: 2.37s
757:	learn: 0.3587121	total: 7.4s	remaining: 2.36s
758:	learn: 0.3586190	total: 7.41s	remaining: 2.35s
759:	learn: 0.3585910	total: 7.42s	remaining: 2.34s
760:	learn: 0.3585281	total: 7.43s	remaining: 2.33s
761:	learn: 0.3584541	total: 7.44s	remaining: 2.32s
762:	learn: 0.3583817	total: 7.45s	remaining: 2.31s
763:	learn: 0.3582717	total: 7.46s	remaining: 2.31s
764:	learn: 0.3582014	total: 7.47s	remaining: 2.29s
765:	learn: 0.3581135	total: 7.48s	remaining: 2.29s
766:	learn: 0.3580422	total: 7.49s	remaining: 2.27s
767:	learn: 0.3580028	total: 7.5s	remaining: 2.27s
768:	learn: 0.3579029	total: 7.51s	remaining: 2.26s
769:	learn: 0.3578433	total: 7.52s	remaining: 2.25s
770:	learn: 0.3577708	total: 7.53s	remaining: 2.24s
771:	learn: 0.3576951	total: 7.54s	remaining: 2.23s
772:	learn: 0.3576543	total: 7.55s	remaining: 2.22s
773:	learn: 0.3575687	total: 7.56s	remaining: 2.21s
774:	learn: 0.3574930	total: 7.57s	remaining: 2.2s
775:	learn: 0.3574348	total: 7.58s	remaining: 2.19s
776:	learn: 0.3573571	total: 7.58s	remaining: 2.18s
777:	learn: 0.3572494	total: 7.59s	remaining: 2.17s
778:	learn: 0.3571783	total: 7.61s	remaining: 2.16s

779:	learn: 0.3571315	total: 7.61s	remaining: 2.15s
780:	learn: 0.3570743	total: 7.62s	remaining: 2.14s
781:	learn: 0.3569598	total: 7.63s	remaining: 2.13s
782:	learn: 0.3569115	total: 7.64s	remaining: 2.12s
783:	learn: 0.3568643	total: 7.65s	remaining: 2.11s
784:	learn: 0.3568180	total: 7.66s	remaining: 2.1s
785:	learn: 0.3567572	total: 7.67s	remaining: 2.09s
786:	learn: 0.3567016	total: 7.68s	remaining: 2.08s
787:	learn: 0.3566025	total: 7.69s	remaining: 2.07s
788:	learn: 0.3564632	total: 7.7s	remaining: 2.06s
789:	learn: 0.3563520	total: 7.71s	remaining: 2.05s
790:	learn: 0.3562514	total: 7.72s	remaining: 2.04s
791:	learn: 0.3561672	total: 7.73s	remaining: 2.03s
792:	learn: 0.3561234	total: 7.74s	remaining: 2.02s
793:	learn: 0.3560382	total: 7.75s	remaining: 2.01s
794:	learn: 0.3559667	total: 7.76s	remaining: 2s
795:	learn: 0.3559050	total: 7.77s	remaining: 1.99s
796:	learn: 0.3558556	total: 7.78s	remaining: 1.98s
797:	learn: 0.3557890	total: 7.79s	remaining: 1.97s
798:	learn: 0.3557156	total: 7.8s	remaining: 1.96s
799:	learn: 0.3555424	total: 7.81s	remaining: 1.95s
800:	learn: 0.3554805	total: 7.82s	remaining: 1.94s
801:	learn: 0.3554137	total: 7.83s	remaining: 1.93s
802:	learn: 0.3552879	total: 7.84s	remaining: 1.92s
803:	learn: 0.3551807	total: 7.85s	remaining: 1.91s
804:	learn: 0.3551115	total: 7.86s	remaining: 1.9s
805:	learn: 0.3550727	total: 7.86s	remaining: 1.89s
806:	learn: 0.3549897	total: 7.87s	remaining: 1.88s
807:	learn: 0.3549317	total: 7.88s	remaining: 1.87s
808:	learn: 0.3548415	total: 7.89s	remaining: 1.86s
809:	learn: 0.3547751	total: 7.9s	remaining: 1.85s
810:	learn: 0.3547043	total: 7.91s	remaining: 1.84s
811:	learn: 0.3546292	total: 7.92s	remaining: 1.83s
812:	learn: 0.3545412	total: 7.93s	remaining: 1.82s
813:	learn: 0.3544350	total: 7.94s	remaining: 1.81s
814:	learn: 0.3543581	total: 7.95s	remaining: 1.8s
815:	learn: 0.3542832	total: 7.96s	remaining: 1.79s
816:	learn: 0.3542351	total: 7.97s	remaining: 1.78s
817:	learn: 0.3541422	total: 7.98s	remaining: 1.77s
818:	learn: 0.3540810	total: 7.99s	remaining: 1.77s
819:	learn: 0.3540081	total: 8s	remaining: 1.76s
820:	learn: 0.3539211	total: 8.01s	remaining: 1.75s
821:	learn: 0.3538391	total: 8.02s	remaining: 1.74s
822:	learn: 0.3538298	total: 8.03s	remaining: 1.73s
823:	learn: 0.3537894	total: 8.04s	remaining: 1.72s
824:	learn: 0.3537009	total: 8.05s	remaining: 1.71s
825:	learn: 0.3536692	total: 8.06s	remaining: 1.7s
826:	learn: 0.3535799	total: 8.07s	remaining: 1.69s
827:	learn: 0.3535397	total: 8.08s	remaining: 1.68s
828:	learn: 0.3534769	total: 8.09s	remaining: 1.67s
829:	learn: 0.3534221	total: 8.1s	remaining: 1.66s
830:	learn: 0.3533296	total: 8.11s	remaining: 1.65s
831:	learn: 0.3532395	total: 8.12s	remaining: 1.64s
832:	learn: 0.3531024	total: 8.13s	remaining: 1.63s
833:	learn: 0.3529967	total: 8.14s	remaining: 1.62s
834:	learn: 0.3528855	total: 8.15s	remaining: 1.61s
835:	learn: 0.3528314	total: 8.16s	remaining: 1.6s
836:	learn: 0.3527670	total: 8.17s	remaining: 1.59s
837:	learn: 0.3526719	total: 8.18s	remaining: 1.58s
838:	learn: 0.3526150	total: 8.19s	remaining: 1.57s

839:	learn: 0.3525344	total: 8.2s	remaining: 1.56s
840:	learn: 0.3524834	total: 8.21s	remaining: 1.55s
841:	learn: 0.3524530	total: 8.22s	remaining: 1.54s
842:	learn: 0.3524212	total: 8.23s	remaining: 1.53s
843:	learn: 0.3523531	total: 8.23s	remaining: 1.52s
844:	learn: 0.3522914	total: 8.24s	remaining: 1.51s
845:	learn: 0.3521749	total: 8.25s	remaining: 1.5s
846:	learn: 0.3521056	total: 8.26s	remaining: 1.49s
847:	learn: 0.3520107	total: 8.27s	remaining: 1.48s
848:	learn: 0.3519053	total: 8.28s	remaining: 1.47s
849:	learn: 0.3518549	total: 8.29s	remaining: 1.46s
850:	learn: 0.3518043	total: 8.3s	remaining: 1.45s
851:	learn: 0.3517121	total: 8.31s	remaining: 1.44s
852:	learn: 0.3515945	total: 8.32s	remaining: 1.43s
853:	learn: 0.3515091	total: 8.33s	remaining: 1.42s
854:	learn: 0.3514279	total: 8.34s	remaining: 1.41s
855:	learn: 0.3513306	total: 8.35s	remaining: 1.4s
856:	learn: 0.3512896	total: 8.36s	remaining: 1.39s
857:	learn: 0.3511772	total: 8.37s	remaining: 1.39s
858:	learn: 0.3511134	total: 8.38s	remaining: 1.38s
859:	learn: 0.3509920	total: 8.39s	remaining: 1.36s
860:	learn: 0.3509196	total: 8.4s	remaining: 1.36s
861:	learn: 0.3508474	total: 8.41s	remaining: 1.35s
862:	learn: 0.3507713	total: 8.42s	remaining: 1.34s
863:	learn: 0.3506665	total: 8.43s	remaining: 1.33s
864:	learn: 0.3505935	total: 8.44s	remaining: 1.32s
865:	learn: 0.3505297	total: 8.45s	remaining: 1.31s
866:	learn: 0.3504547	total: 8.46s	remaining: 1.3s
867:	learn: 0.3503849	total: 8.47s	remaining: 1.29s
868:	learn: 0.3503008	total: 8.48s	remaining: 1.28s
869:	learn: 0.3502058	total: 8.49s	remaining: 1.27s
870:	learn: 0.3501418	total: 8.5s	remaining: 1.26s
871:	learn: 0.3500624	total: 8.51s	remaining: 1.25s
872:	learn: 0.3499849	total: 8.52s	remaining: 1.24s
873:	learn: 0.3499229	total: 8.53s	remaining: 1.23s
874:	learn: 0.3498792	total: 8.54s	remaining: 1.22s
875:	learn: 0.3498385	total: 8.55s	remaining: 1.21s
876:	learn: 0.3497594	total: 8.56s	remaining: 1.2s
877:	learn: 0.3496819	total: 8.56s	remaining: 1.19s
878:	learn: 0.3496314	total: 8.57s	remaining: 1.18s
879:	learn: 0.3495385	total: 8.58s	remaining: 1.17s
880:	learn: 0.3494538	total: 8.59s	remaining: 1.16s
881:	learn: 0.3493806	total: 8.6s	remaining: 1.15s
882:	learn: 0.3492701	total: 8.61s	remaining: 1.14s
883:	learn: 0.3492499	total: 8.62s	remaining: 1.13s
884:	learn: 0.3492010	total: 8.63s	remaining: 1.12s
885:	learn: 0.3491033	total: 8.64s	remaining: 1.11s
886:	learn: 0.3490311	total: 8.65s	remaining: 1.1s
887:	learn: 0.3489909	total: 8.66s	remaining: 1.09s
888:	learn: 0.3489468	total: 8.67s	remaining: 1.08s
889:	learn: 0.3488473	total: 8.68s	remaining: 1.07s
890:	learn: 0.3487627	total: 8.69s	remaining: 1.06s
891:	learn: 0.3486368	total: 8.7s	remaining: 1.05s
892:	learn: 0.3485700	total: 8.71s	remaining: 1.04s
893:	learn: 0.3484601	total: 8.72s	remaining: 1.03s
894:	learn: 0.3483860	total: 8.73s	remaining: 1.02s
895:	learn: 0.3482792	total: 8.74s	remaining: 1.01s
896:	learn: 0.3482400	total: 8.75s	remaining: 1s
897:	learn: 0.3481554	total: 8.76s	remaining: 995ms
898:	learn: 0.3480840	total: 8.77s	remaining: 985ms

899:	learn: 0.3479869	total: 8.78s	remaining: 975ms
900:	learn: 0.3479078	total: 8.79s	remaining: 966ms
901:	learn: 0.3478767	total: 8.8s	remaining: 956ms
902:	learn: 0.3478062	total: 8.81s	remaining: 946ms
903:	learn: 0.3477423	total: 8.81s	remaining: 936ms
904:	learn: 0.3476605	total: 8.82s	remaining: 926ms
905:	learn: 0.3476018	total: 8.83s	remaining: 917ms
906:	learn: 0.3475342	total: 8.84s	remaining: 907ms
907:	learn: 0.3474748	total: 8.85s	remaining: 897ms
908:	learn: 0.3474488	total: 8.86s	remaining: 887ms
909:	learn: 0.3473929	total: 8.87s	remaining: 877ms
910:	learn: 0.3472931	total: 8.88s	remaining: 868ms
911:	learn: 0.3471964	total: 8.89s	remaining: 858ms
912:	learn: 0.3471403	total: 8.9s	remaining: 848ms
913:	learn: 0.3470810	total: 8.91s	remaining: 839ms
914:	learn: 0.3470231	total: 8.92s	remaining: 829ms
915:	learn: 0.3469557	total: 8.93s	remaining: 819ms
916:	learn: 0.3468797	total: 8.94s	remaining: 809ms
917:	learn: 0.3467927	total: 8.95s	remaining: 799ms
918:	learn: 0.3467348	total: 8.96s	remaining: 790ms
919:	learn: 0.3466941	total: 8.97s	remaining: 780ms
920:	learn: 0.3466101	total: 8.98s	remaining: 770ms
921:	learn: 0.3465531	total: 8.99s	remaining: 760ms
922:	learn: 0.3465004	total: 9s	remaining: 751ms
923:	learn: 0.3464326	total: 9.01s	remaining: 741ms
924:	learn: 0.3463013	total: 9.02s	remaining: 731ms
925:	learn: 0.3462573	total: 9.03s	remaining: 721ms
926:	learn: 0.3461467	total: 9.04s	remaining: 712ms
927:	learn: 0.3460712	total: 9.05s	remaining: 702ms
928:	learn: 0.3459898	total: 9.06s	remaining: 692ms
929:	learn: 0.3458816	total: 9.07s	remaining: 682ms
930:	learn: 0.3457798	total: 9.08s	remaining: 673ms
931:	learn: 0.3457056	total: 9.09s	remaining: 663ms
932:	learn: 0.3456404	total: 9.1s	remaining: 653ms
933:	learn: 0.3455645	total: 9.11s	remaining: 643ms
934:	learn: 0.3455049	total: 9.12s	remaining: 634ms
935:	learn: 0.3454685	total: 9.12s	remaining: 624ms
936:	learn: 0.3453931	total: 9.13s	remaining: 614ms
937:	learn: 0.3453452	total: 9.14s	remaining: 604ms
938:	learn: 0.3453444	total: 9.15s	remaining: 595ms
939:	learn: 0.3452935	total: 9.16s	remaining: 585ms
940:	learn: 0.3452073	total: 9.17s	remaining: 575ms
941:	learn: 0.3451694	total: 9.18s	remaining: 565ms
942:	learn: 0.3450773	total: 9.19s	remaining: 556ms
943:	learn: 0.3449398	total: 9.21s	remaining: 546ms
944:	learn: 0.3448665	total: 9.21s	remaining: 536ms
945:	learn: 0.3447874	total: 9.22s	remaining: 527ms
946:	learn: 0.3447514	total: 9.23s	remaining: 517ms
947:	learn: 0.3446855	total: 9.24s	remaining: 507ms
948:	learn: 0.3445533	total: 9.25s	remaining: 497ms
949:	learn: 0.3444505	total: 9.26s	remaining: 488ms
950:	learn: 0.3444003	total: 9.27s	remaining: 478ms
951:	learn: 0.3443696	total: 9.28s	remaining: 468ms
952:	learn: 0.3443455	total: 9.29s	remaining: 458ms
953:	learn: 0.3442648	total: 9.3s	remaining: 449ms
954:	learn: 0.3442173	total: 9.31s	remaining: 439ms
955:	learn: 0.3441151	total: 9.32s	remaining: 429ms
956:	learn: 0.3440497	total: 9.33s	remaining: 419ms
957:	learn: 0.3440012	total: 9.36s	remaining: 410ms
958:	learn: 0.3439152	total: 9.37s	remaining: 401ms

959:	learn: 0.3438284	total: 9.38s	remaining: 391ms
960:	learn: 0.3437245	total: 9.39s	remaining: 381ms
961:	learn: 0.3436757	total: 9.4s	remaining: 371ms
962:	learn: 0.3436045	total: 9.41s	remaining: 362ms
963:	learn: 0.3435039	total: 9.42s	remaining: 352ms
964:	learn: 0.3434011	total: 9.44s	remaining: 342ms
965:	learn: 0.3433159	total: 9.45s	remaining: 333ms
966:	learn: 0.3432238	total: 9.46s	remaining: 323ms
967:	learn: 0.3431730	total: 9.48s	remaining: 313ms
968:	learn: 0.3431256	total: 9.49s	remaining: 304ms
969:	learn: 0.3430910	total: 9.51s	remaining: 294ms
970:	learn: 0.3430498	total: 9.54s	remaining: 285ms
971:	learn: 0.3429800	total: 9.56s	remaining: 275ms
972:	learn: 0.3429151	total: 9.59s	remaining: 266ms
973:	learn: 0.3428682	total: 9.62s	remaining: 257ms
974:	learn: 0.3427736	total: 9.64s	remaining: 247ms
975:	learn: 0.3426995	total: 9.67s	remaining: 238ms
976:	learn: 0.3426068	total: 9.69s	remaining: 228ms
977:	learn: 0.3425173	total: 9.71s	remaining: 219ms
978:	learn: 0.3424791	total: 9.73s	remaining: 209ms
979:	learn: 0.3424491	total: 9.75s	remaining: 199ms
980:	learn: 0.3423677	total: 9.76s	remaining: 189ms
981:	learn: 0.3422543	total: 9.78s	remaining: 179ms
982:	learn: 0.3421684	total: 9.79s	remaining: 169ms
983:	learn: 0.3420703	total: 9.81s	remaining: 159ms
984:	learn: 0.3419534	total: 9.82s	remaining: 150ms
985:	learn: 0.3418881	total: 9.83s	remaining: 140ms
986:	learn: 0.3418236	total: 9.84s	remaining: 130ms
987:	learn: 0.3417437	total: 9.85s	remaining: 120ms
988:	learn: 0.3416728	total: 9.87s	remaining: 110ms
989:	learn: 0.3415872	total: 9.88s	remaining: 99.8ms
990:	learn: 0.3415324	total: 9.89s	remaining: 89.8ms
991:	learn: 0.3414871	total: 9.9s	remaining: 79.8ms
992:	learn: 0.3414424	total: 9.91s	remaining: 69.9ms
993:	learn: 0.3413376	total: 9.92s	remaining: 59.9ms
994:	learn: 0.3413031	total: 9.93s	remaining: 49.9ms
995:	learn: 0.3411767	total: 9.94s	remaining: 39.9ms
996:	learn: 0.3411159	total: 9.95s	remaining: 29.9ms
997:	learn: 0.3410251	total: 9.96s	remaining: 20ms
998:	learn: 0.3409859	total: 9.97s	remaining: 9.98ms
999:	learn: 0.3409243	total: 9.98s	remaining: 0us

Out[135...] CatBoostClassifier()

```
In [136...] y_pred_9=LGBM_M8.predict(X_test)
y_pred_9[:10]
```

Out[136...] array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)

```
In [137...] # Model Evaluation
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
In [138...] confusion_matrix(y_test, y_pred_9)
```

Out[138...] array([[6705, 353],
[1230, 712]], dtype=int64)

```
In [139...] accuracy_score(y_test, y_pred_8)
```

Out[139... 0.8241111111111111

```
In [140... print(classification_report(y_test,y_pred_8))
```

	precision	recall	f1-score	support
0	0.84	0.95	0.89	7058
1	0.67	0.37	0.47	1942
accuracy			0.82	9000
macro avg	0.76	0.66	0.68	9000
weighted avg	0.81	0.82	0.80	9000

TASK 5: Hyperparameter Tuning

1. Tune at least 3 models by using GridSearchCV or RandomizedSearchCV

a. SVM

b. Random Forest

c. XGBoost

2. Compare these three models Before tuning vs After tuning

SVM(Hyperparameter Tuning)

```
In [318... from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC
```

```
In [320... svm_M3=SVC()
```

```
In [324... hyperparameter_svm={
    'C':[0.01,0.1,0.3,0.4,0.5,1],
    'kernel':['rbf','linear','poly','sigmoid']}
```

```
In [326... svm_grid=GridSearchCV(estimator=SVC(),
    param_grid=hyperparameter_svm,cv=3)
```

```
In [328... svm_grid.fit(X_train_scaled,y_train)
```

```
Out[328... ▸ GridSearchCV
▸ estimator: SVC
    ▸ SVC
```

```
In [335... y_pred_svm=svm_grid.predict(X_test_scaled)
y_pred_svm[:10]
```

```
Out[335... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
In [337... svm_grid.best_estimator_
```

Out[337...

▼ SVC
SVC(C=1)

In [339...

```
svm_grid.best_params_
```

Out[339...

```
{'C': 1, 'kernel': 'rbf'}
```

In [341...

```
svm_grid.best_score_
```

Out[341...

```
0.818
```

In [343...

```
# Model Evaluation  
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

In [345...

```
confusion_matrix(y_test, y_pred_svm)
```

Out[345...

```
array([[6774, 284],  
       [1295, 647]], dtype=int64)
```

In [347...

```
print(classification_report(y_test, y_pred_svm))
```

	precision	recall	f1-score	support
0	0.84	0.96	0.90	7058
1	0.69	0.33	0.45	1942
accuracy			0.82	9000
macro avg	0.77	0.65	0.67	9000
weighted avg	0.81	0.82	0.80	9000

Random Forest(Hyperparameter Tuning)

In [155...

```
from sklearn.model_selection import GridSearchCV  
from sklearn.ensemble import RandomForestClassifier
```

In [156...

```
hyperparameter_Random={ 'criterion': ['gini', 'log_loss', 'entropy'],  
                        'max_depth': [5, 10],  
                        'max_features': ['sqrt', 'log2'],  
                        'n_estimators': [100, 500]}
```

In [157...

```
Random_grid=GridSearchCV(estimator=RandomForestClassifier(),  
                        param_grid=hyperparameter_Random, cv=3,  
                        n_jobs=-1)
```

In [158...

```
Random_grid.fit(X_train, y_train)
```

Out[158...

```
GridSearchCV  
estimator: RandomForestClassifier  
RandomForestClassifier
```

```
In [278... y_pred_rndm=Random_grid.predict(X_test)
y_pred_rndm[:10]
```

```
Out[278... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int64)
```

```
In [280... #Model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_report
```

```
In [282... confusion_matrix(y_test,y_pred_rndm)
```

```
Out[282... array([[6751,  307],
        [1269,  673]], dtype=int64)
```

```
In [284... accuracy_score(y_test,y_pred_rndm)
```

```
Out[284... 0.8248888888888889
```

```
In [286... print(classification_report(y_test,y_pred_rndm))
```

	precision	recall	f1-score	support
0	0.84	0.96	0.90	7058
1	0.69	0.35	0.46	1942
accuracy			0.82	9000
macro avg	0.76	0.65	0.68	9000
weighted avg	0.81	0.82	0.80	9000

```
In [290... Random_grid.best_params_
```

```
Out[290... {'criterion': 'log_loss',
'max_depth': 10,
'max_features': 'sqrt',
'n_estimators': 500}
```

XGBoost

```
In [295... from xgboost import XGBClassifier
```

```
In [297... hyperparameters_xg={
    'n_estimators':[100,200],
    'max_depth':[3,6],
    'learning_rate':[0.01,0.1]
}
```

```
In [299... xgb_grid=GridSearchCV(estimator=XGBClassifier(),param_grid=hyperparameters_xg,cv
```

```
In [301... xgb_grid.fit(X_train,y_train)
```

```
Out[301...
└─ GridSearchCV
  └─ estimator: XGBClassifier
    └─ XGBClassifier
```



```
In [305... y_pred_xgb=xgb_grid.predict(X_test)
y_pred_xgb[:10]
```

```
Out[305... array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [307... #Model Evaluation
from sklearn.metrics import confusion_matrix,accuracy_score,classification_report
```

```
In [309... confusion_matrix(y_test,y_pred_xgb)
```

```
Out[309... array([[6773, 285],
       [1290, 652]], dtype=int64)
```

```
In [311... accuracy_score(y_test,y_pred_xgb)
```

```
Out[311... 0.825
```

```
In [313... print(classification_report(y_test,y_pred_xgb))
```

	precision	recall	f1-score	support
0	0.84	0.96	0.90	7058
1	0.70	0.34	0.45	1942
accuracy			0.82	9000
macro avg	0.77	0.65	0.67	9000
weighted avg	0.81	0.82	0.80	9000

```
In [315... xgb_grid.best_params_
```

```
Out[315... {'learning_rate': 0.01, 'max_depth': 6, 'n_estimators': 200}
```

Compare these three models Before tuning vs After tuning

```
In [351... results=pd.DataFrame({
    'model':['SVM','SVM','Random Forest','Random Forest','XGBoost','XGBoost'],
    'phase':['Before tuning','After tuning','Before tuning','After tuning','Before tuning','After tuning'],
    'Accuracy':[0.82,0.82,0.82,0.82,0.81,0.82],
    'precision':[0.81,0.81,0.80,0.81,0.79,0.81],
    'Recall':[0.82,0.82,0.82,0.82,0.81,0.82 ],
    'f1-score':[0.80,0.80,0.80, 0.80, 0.79, 0.80]})
results
```

```
Out[351...
```

	model	phase	Accuracy	precision	Recall	f1-score
0	SVM	Before tuning	0.82	0.81	0.82	0.80
1	SVM	After tuning	0.82	0.81	0.82	0.80
2	Random Forest	Before tuning	0.82	0.80	0.82	0.80
3	Random Forest	After tuning	0.82	0.81	0.82	0.80
4	XGBoost	Before tuning	0.81	0.79	0.81	0.79
5	XGBoost	After tuning	0.82	0.81	0.82	0.80

TASK 6: Model Comparison & Selection

1. Create the following comparison table for all 9 Models:

Model Accuracy Precision Recall F1

2. Provide the following Insights:

d. Which model performs best?

e. Is there any overfitting or underfitting? (plot graphs for the final model)

f. Which metric influenced your final decision?

g. Analyze: -

Does payment delay history influence risk more? -

Does credit limit affect payment behavior?

```
In [378... #Create the following comparison table for all 9 Models:
comparison_table=pd.DataFrame({
    'model':['Logistic Regression','KNN','SVM','Decision Tree','Random Forest','
    'Accuracy':[0.82,0.82,0.82,0.73,0.82,0.82,0.81,0.82,0.82],
    'precision':[0.80,0.80,0.81,0.74,0.80,0.80,0.79,0.81,0.81],
    'Recall':[0.82,0.82,0.82,0.73,0.82,0.82,0.81,0.82,0.82 ],
    'f1-score':[0.78,0.78,0.80, 0.73, 0.80, 0.79,0.79,0.80,0.80]})
comparison_table
```

```
Out[378...
      model  Accuracy  precision  Recall  f1-score
0  Logistic Regression    0.82    0.80    0.82    0.78
1           KNN          0.82    0.80    0.82    0.78
2           SVM          0.82    0.81    0.82    0.80
3  Decision Tree          0.73    0.74    0.73    0.73
4  Random Forest          0.82    0.80    0.82    0.80
5      AdaBoost          0.82    0.80    0.82    0.79
6      XGBoost          0.81    0.79    0.81    0.79
7      LightGBM          0.82    0.81    0.82    0.80
8      CatBoost          0.82    0.81    0.82    0.80
```

Which model performs best?

Based on the model comparison, SVM, Random Forest, LightGBM, and CatBoost achieved the highest performance with an accuracy of 0.82 and an F1-score of 0.80. These models provide a better balance between precision and recall compared to the other algorithms. Among them, LightGBM and CatBoost are considered the best-

performing models because they maintain high precision, recall, and F1-score while effectively handling complex patterns in the dataset.

Is there any overfitting or underfitting? (plot graphs for the final model)

In [392...

```
from sklearn.model_selection import learning_curve
import numpy as np
import matplotlib.pyplot as plt

# choose your final model
final_model = LGBM_M8 # example (change if needed)

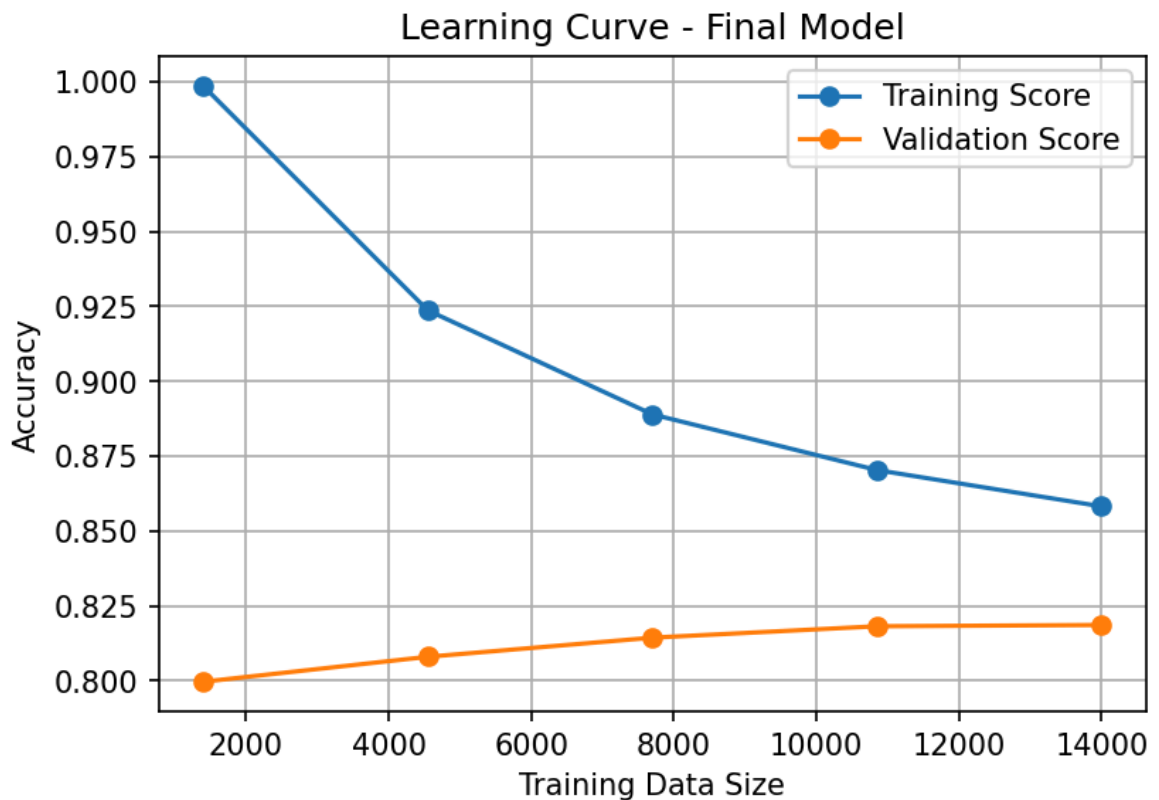
# generate learning curve data
train_sizes, train_scores, val_scores = learning_curve(
    final_model,
    X_train,
    y_train,
    cv=3,
    scoring='accuracy',
    n_jobs=-1,
    train_sizes=np.linspace(0.1,1.0,5)
)

# calculate mean scores
train_mean = np.mean(train_scores, axis=1)
val_mean = np.mean(val_scores, axis=1)

# plot graph
plt.figure(figsize=(6,4), dpi=150)
plt.plot(train_sizes, train_mean, marker='o', label='Training Score')
plt.plot(train_sizes, val_mean, marker='o', label='Validation Score')

plt.xlabel("Training Data Size")
plt.ylabel("Accuracy")
plt.title("Learning Curve - Final Model")
plt.legend()
plt.grid(True)

plt.show()
```



The learning curve shows the relationship between training accuracy and validation accuracy as the training data size increases. Initially, the training accuracy is very high while the validation accuracy is lower, indicating slight overfitting when the model is trained on a small amount of data. As the training data increases, the training accuracy decreases and the validation accuracy improves. The gap between the two curves becomes smaller, which indicates that the model generalizes well and does not show significant overfitting or underfitting.

Which metric influenced your final decision?

The F1-score influenced the final decision because it balances both precision and recall and is more reliable for imbalanced datasets like credit default prediction.

Does payment delay history influence risk more? -

Customers with a history of payment delays are more likely to default. Therefore, payment delay variables are strong predictors of credit risk.

Does credit limit affect payment behavior?

Credit limit influences payment behavior, as customers with higher credit limits tend to show more stable repayment patterns compared to those with lower limits.

In []: